SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 40 — Data Import & Upload Framework

Property	Value
Document ID	SMS-FSD-ADD-040
Version	1.0
Date	2026-10-11
Status	Draft
Author	System Architect
Classification	Internal — Development
Depends On	SMS-FSD-ADD-001 (Core), SMS-FSD-ADD-012 (Finance), SMS-FSD-ADD-038 (POS)
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18 / ClosedXML / CsvHelper / Hangfire
Change Items	9 changes (C1–C9)
Impact	10 migrations, 7 new tables, 17 API endpoints, 6 permissions, 30 test scenarios

---

Table of Contents

1. Purpose & Scope
2. C1 — Import Profile Registry
3. C2 — Import Profile Fields
4. C3 — Import Session Tracking
5. C4 — External Reference Registry
6. C5 — Excel Template Engine
7. C6 — Column Mapping & Lookup Resolution
8. C7 — Validation Engine
9. C8 — Import Execution Pipeline
10. C9 — Opening Balance Import
11. Import UI Screens
12. Database Migrations
13. API Endpoints
14. Permission Claims
15. Business Rules
16. Test Scenarios
17. Development Phases
18. Future Enhancements
19. Appendix A — Entity Relationship Summary
20. Appendix B — Seeded Import Profile Fields

21. Appendix C — Import Flow Decision Matrix

1. Purpose & Scope

This addendum introduces a universal data import and upload framework for SMS. It enables organizations to bring data from legacy ERP systems, accounting software, spreadsheets, or any external source into SMS via structured Excel/CSV file upload. The framework provides downloadable templates per entity type, intelligent column mapping, comprehensive validation with error reporting, batch import execution via Hangfire, and a central external reference registry for upsert (create-or-update) operations. The design follows Odoo's import model — every importable entity has a system-defined import profile with a downloadable template, and the column mapping wizard lets users map their source file's columns to SMS fields without reformatting their data first.

1.1 Design Principles

- **Template-First** — Every importable entity has a downloadable Excel template with correct headers, data validation dropdowns (for lookup fields like category, unit, warehouse), sample rows, and column-level instructions. Users who start from the template skip the column mapping step entirely.
- **Map Any Source** — Users uploading from legacy systems or existing spreadsheets get an intelligent column mapping wizard. Headers are auto-matched by name similarity, and unmapped columns can be manually assigned. Saved mapping presets let users reuse mappings for recurring imports.
- **Validate Before Write** — A dry-run validation pass checks every row for required fields, data types, referential integrity (does that category exist?), uniqueness (duplicate SKU?), and business rules before any database write. Invalid rows are flagged with row number + field + error — the user can fix and re-upload only the failed rows.
- **External ID for Upsert** — Every import row can include an optional `external_id` column — a unique identifier from the source system. The `import.ExternalReferences` table maps these to SMS entity IDs. On subsequent imports, rows with existing external IDs update the record instead of creating a duplicate. This is the same concept as Odoo's `ir.model.data` XML ID.
- **Batch Processing** — Large files (configurable threshold, default 500+ rows) run as Hangfire background jobs with real-time progress tracking. Small files process synchronously for instant results.
- **Opening Balances as Import Type** — Opening stock, customer balances, supplier balances, and GL opening balances are special import types that generate the appropriate journal entries and stock movements behind the scenes, handling the accounting correctly during initial setup.
- **Dependency-Aware** — Import profiles have a defined dependency order (categories before products, warehouses before stock). The UI guides users through the recommended import sequence during initial data migration.

1.2 Target Scope

Included	Excluded (Future)
Excel (.xlsx) and CSV file import	Direct database-to-database migration tool
Downloadable templates per entity type	API-based bulk import (ETL connectors)
Column mapping wizard with auto-match	Scheduled/recurring auto-import
Validation with detailed error report	Real-time API streaming import
Batch Hangfire processing for large files	Bi-directional sync with legacy systems
External ID registry for upsert	Custom entity import (user-defined tables)
Saved mapping presets	Multi-file relationship import (single upload)
Opening stock/balance import	Image/document attachment import
16 seeded entity import profiles	Legacy .xls (pre-2007) format
Error report download as Excel	Undo/rollback of individual rows

1.3 Importable Entity Types

Entity Type	Enum	Target	Match Field(s)	Dependency
Customer	0	customers.Customers	code, email	—
Supplier	1	procurement.Suppliers	code, email	—
Product	2	inventory.Products	sku	ProductCategory, Unit
Product Category	3	inventory.ProductCategories	code	— (self-referencing parent)
Unit of Measure	4	inventory.Units	code	—
Warehouse	5	inventory.Warehouses	code	—
Warehouse Location	6	inventory.WarehouseLocations	code	Warehouse
Chart of Accounts	7	finance.Accounts	account_code	— (self-referencing parent)
Tax Code	8	finance.TaxCodes	code	Account
Contact	9	core.Contacts	email	Customer or Supplier
Price List Item	10	inventory.PriceListItems	product_sku + price_list_code	Product, PriceList
Employee	11	hr.Employees	employee_code	—
Opening Stock	12	(special handler)	product_sku + warehouse_code	Product, Warehouse
Customer Opening Balance	13	(special handler)	customer_code	Customer, Account
Supplier Opening Balance	14	(special handler)	supplier_code	Supplier, Account
GL Opening Balance	15	(special handler)	account_code	Account

1.4 Recommended Import Sequence

For initial data migration from a legacy system, entities should be imported in this order (dependency chain):

```
Step 1:  Units of Measure (no dependencies)
Step 2:  Product Categories (self-referencing — import parents first)
Step 3:  Products (depends on Category + Unit)
Step 4:  Warehouses (no dependencies)
Step 5:  Warehouse Locations (depends on Warehouse)
Step 6:  Chart of Accounts (self-referencing — import parents first)
Step 7:  Tax Codes (depends on Account)
Step 8:  Customers (no dependencies)
Step 9:  Suppliers (no dependencies)
Step 10: Contacts (depends on Customer or Supplier)
Step 11: Employees (no dependencies — can run with Step 1)
Step 12: Price List Items (depends on Product)
Step 13: Opening Stock (depends on Product + Warehouse)
Step 14: Customer Opening Balances (depends on Customer + Account)
Step 15: Supplier Opening Balances (depends on Supplier + Account)
Step 16: GL Opening Balances (depends on Account)
```

1.5 Changes Summary

Change ID	Title	Category
C1	Import Profile Registry	New table — `import.ImportProfiles`
C2	Import Profile Fields	New table — `import.ImportProfileFields`
C3	Import Session Tracking	New tables — `import.ImportSessions`, `import.ImportColumnMappings`, `import.ImportRows`
C4	External Reference Registry	New table — `import.ExternalReferences`
C5	Excel Template Engine	Service — ClosedXML template generation with data validation
C6	Column Mapping & Lookup Resolution	Service — auto-mapping + lookup resolver
C7	Validation Engine	Service — type, required, regex, lookup, uniqueness validation
C8	Import Execution Pipeline	Service — batch upsert, entity handlers, Hangfire
C9	Opening Balance Import	Service — stock, AR, AP, GL opening balance handlers

---

2. C1 — Import Profile Registry

2.1 Rationale

An import profile defines one importable entity type — its name, the target table, how duplicates are detected, and the recommended import order. System profiles are seeded during installation for all 16 entity types. Organizations can create custom profiles to adjust duplicate-handling behavior for their specific needs but cannot change the underlying entity structure.

2.2 Entity

2.2.1 `import.ImportProfiles`

Column	Type	Constraints	Description
`import_profile_id`	BIGINT	PK, IDENTITY	Auto-increment PK
`org_id`	BIGINT	NULL, FK → tenant.Organizations	NULL = system-wide profile (seeded). Non-NULL = org-specific custom profile
`entity_type`	TINYINT	NOT NULL	Enum: 0=Customer, 1=Supplier, 2=Product, 3=ProductCategory, 4=Unit, 5=Warehouse, 6=WarehouseLocation, 7=Account, 8=TaxCode, 9=Contact, 10=PriceListItem, 11=Employee, 12=OpeningStock, 13=CustomerOpeningBalance, 14=SupplierOpeningBalance, 15=GLOpeningBalance
`name`	NVARCHAR(200)	NOT NULL	Display name: "Product Import", "Customer Import"
`description`	NVARCHAR(500)	NULL	Description shown in the import wizard
`target_schema`	VARCHAR(50)	NOT NULL	Target DB schema: "inventory", "customers", "finance", etc.
`target_table`	VARCHAR(100)	NOT NULL	Target table name: "Products", "Customers", etc. Empty for special handlers (opening balances)
`match_fields`	NVARCHAR(300)	NOT NULL	JSON array of field names for duplicate detection: `["sku"]`, `["code","email"]`
`duplicate_strategy`	TINYINT	NOT NULL, DEFAULT 0	0=Skip, 1=Update, 2=CreateNew, 3=Error
`import_order`	INT	NOT NULL, DEFAULT 0	Recommended sequence in migration wizard (lower = import first)
`max_rows_per_file`	INT	NOT NULL, DEFAULT 50000	Maximum allowed rows per upload
`max_file_size_mb`	INT	NOT NULL, DEFAULT 10	Maximum file size in MB
`is_opening_balance`	BIT	NOT NULL, DEFAULT 0	Whether this is an opening balance import (uses special handler)
`is_system`	BIT	NOT NULL, DEFAULT 1	System-seeded profile (not editable). Custom profiles = 0
`is_active`	BIT	NOT NULL, DEFAULT 1	
`created_at`	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
`created_by`	BIGINT	NULL, FK → auth.Users	NULL for system-seeded profiles
`modified_at`	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
`modified_by`	BIGINT	NULL, FK → auth.Users	

Indexes:

- PK: `import_profile_id`
- UQ: `(org_id, entity_type)` WHERE org_id IS NOT NULL — one custom profile per entity per org
- IX: `(entity_type, is_active)` — lookup by entity type
- IX: `(org_id, is_active, import_order)` — migration wizard sort

Seeded System Profiles (M040.8):

entity_type	name	target_schema	target_table	match_fields	import_order
0	Customer Import	customers	Customers	`["code","email"]`	8
1	Supplier Import	procurement	Suppliers	`["code","email"]`	9
2	Product Import	inventory	Products	`["sku"]`	3
3	Product Category Import	inventory	ProductCategories	`["code"]`	2
4	Unit of Measure Import	inventory	Units	`["code"]`	1
5	Warehouse Import	inventory	Warehouses	`["code"]`	4
6	Warehouse Location Import	inventory	WarehouseLocations	`["code"]`	5
7	Chart of Accounts Import	finance	Accounts	`["account_code"]`	6
8	Tax Code Import	finance	TaxCodes	`["code"]`	7
9	Contact Import	core	Contacts	`["email"]`	10
10	Price List Item Import	inventory	PriceListItems	`["product_sku","price_list_code"]`	12
11	Employee Import	hr	Employees	`["employee_code"]`	11
12	Opening Stock Import	—	—	`["product_sku","warehouse_code"]`	13
13	Customer Opening Balance	—	—	`["customer_code"]`	14
14	Supplier Opening Balance	—	—	`["supplier_code"]`	15
15	GL Opening Balance	—	—	`["account_code"]`	16

3. C2 — Import Profile Fields

3.1 Rationale

Each import profile defines a set of fields that appear as columns in the downloadable template and as targets for column mapping. System profiles have pre-defined fields matching the SMS entity structure. Field definitions include data type, validation rules, and lookup references for resolving names to IDs.

3.2 Entity

3.2.1 `import.ImportProfileFields`

Column	Type	Constraints	Description
`import_profile_field_id`	BIGINT	PK, IDENTITY	Auto-increment PK
`import_profile_id`	BIGINT	NOT NULL, FK → import.ImportProfiles	Parent profile
`field_name`	VARCHAR(100)	NOT NULL	Internal field name matching the target entity property: `sku`, `name`, `sale_price`
`display_name`	NVARCHAR(200)	NOT NULL	Column header in template and mapping UI: "SKU / Code", "Sale Price"
`description`	NVARCHAR(500)	NULL	Help text shown in template header row and mapping tooltip
`field_type`	TINYINT	NOT NULL	0=Text, 1=Integer, 2=Decimal, 3=Date, 4=Boolean, 5=Lookup, 6=Email, 7=Phone, 8=Currency
`is_required`	BIT	NOT NULL, DEFAULT 0	Mandatory field — validation fails if blank
`is_key`	BIT	NOT NULL, DEFAULT 0	Part of the natural key for duplicate detection
`is_template_column`	BIT	NOT NULL, DEFAULT 1	Include in downloadable template. False = system-only field
`max_length`	INT	NULL	Maximum character length for text fields
`min_value`	DECIMAL(18,4)	NULL	Minimum value for numeric fields
`max_value`	DECIMAL(18,4)	NULL	Maximum value for numeric fields
`decimal_places`	TINYINT	NULL	Required decimal precision for numeric fields
`validation_regex`	NVARCHAR(500)	NULL	Regex pattern for validation (e.g., email format, phone format)
`default_value`	NVARCHAR(500)	NULL	Default value if the field is blank in the import file
`lookup_entity_type`	TINYINT	NULL	For field_type=Lookup: which entity type to resolve against
`lookup_match_field`	VARCHAR(100)	NULL	For field_type=Lookup: which field of the lookup entity to match (e.g., "code", "name")
`lookup_dropdown`	BIT	NOT NULL, DEFAULT 0	For field_type=Lookup: include a data validation dropdown in the Excel template
`sort_order`	INT	NOT NULL, DEFAULT 0	Column order in template
`sample_value`	NVARCHAR(200)	NULL	Sample value shown in template's example row

Indexes:

- PK: `import_profile_field_id`
- UQ: `(import_profile_id, field_name)` — one field per name per profile
- IX: `(import_profile_id, sort_order)` — ordered field list

Example: Product Import Profile Fields (seeded via M040.9):

field_name	display_name	field_type	required	is_key	lookup	sort
`external_id`	External ID (Source System)	Text	No	No	—	0
`sku`	SKU / Code	Text	Yes	Yes	—	1
`name`	Product Name	Text	Yes	No	—	2
`description`	Description	Text	No	No	—	3
`category_code`	Category Code	Lookup	No	No	ProductCategory → code	4
`unit_code`	Unit of Measure Code	Lookup	Yes	No	Unit → code	5
`sale_price`	Sale Price	Decimal	No	No	—	6
`cost_price`	Cost Price	Decimal	No	No	—	7
`barcode`	Barcode	Text	No	No	—	8
`weight`	Weight (kg)	Decimal	No	No	—	9
`min_stock_level`	Min Stock Level	Decimal	No	No	—	10
`max_stock_level`	Max Stock Level	Decimal	No	No	—	11
`is_service`	Is Service Item	Boolean	No	No	—	12
`is_active`	Active	Boolean	No	No	—	13

> **Note:** Complete field definitions for all 16 entity types are in **Appendix B**.

4. C3 — Import Session Tracking

4.1 Rationale

Every file upload creates an import session that tracks the entire lifecycle: upload → mapping → validation → execution → completion. Sessions store per-row results so users can review what was imported, what was skipped, and what failed — with downloadable error reports.

4.2 Entities

4.2.1 `import.ImportSessions`

Column	Type	Constraints	Description
`import_session_id`	BIGINT	PK, IDENTITY	Auto-increment PK
`org_id`	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
`import_profile_id`	BIGINT	NOT NULL, FK → import.ImportProfiles	Which entity type being imported
`file_name`	NVARCHAR(500)	NOT NULL	Original uploaded file name
`file_size_bytes`	BIGINT	NOT NULL	File size
`file_type`	TINYINT	NOT NULL	0=XLSX, 1=CSV
`sheet_name`	NVARCHAR(200)	NULL	For Excel: which sheet was selected (NULL = first sheet)
`duplicate_strategy`	TINYINT	NOT NULL	0=Skip, 1=Update, 2=CreateNew, 3=Error. Copied from profile default, overridable per session
`total_rows`	INT	NOT NULL, DEFAULT 0	Total data rows (excluding header)
`valid_rows`	INT	NOT NULL, DEFAULT 0	Rows that passed validation
`invalid_rows`	INT	NOT NULL, DEFAULT 0	Rows that failed validation
`imported_rows`	INT	NOT NULL, DEFAULT 0	Successfully imported rows
`updated_rows`	INT	NOT NULL, DEFAULT 0	Updated via upsert
`skipped_rows`	INT	NOT NULL, DEFAULT 0	Skipped (duplicate strategy = Skip)
`error_rows`	INT	NOT NULL, DEFAULT 0	Failed during import execution
`status`	TINYINT	NOT NULL, DEFAULT 0	0=Uploaded, 1=Mapping, 2=Validating, 3=Validated, 4=Importing, 5=Completed, 6=CompletedWithErrors, 7=Failed, 8=Cancelled
`validation_started_at`	DATETIME2(2)	NULL	When validation pass started
`validation_completed_at`	DATETIME2(2)	NULL	When validation pass finished
`import_started_at`	DATETIME2(2)	NULL	When import execution started
`import_completed_at`	DATETIME2(2)	NULL	When import execution finished
`error_message`	NVARCHAR(MAX)	NULL	Session-level error (file parse failure, etc.)
`hangfire_job_id`	VARCHAR(200)	NULL	Hangfire job ID for background processing

Column	Type	Constraints	Description
`saved_mapping_name`	NVARCHAR(200)	NULL	If user saved this session's column mapping as a reusable preset
`notes`	NVARCHAR(1000)	NULL	User notes about this import ("Q3 customer list from SAP")
`created_at`	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
`created_by`	BIGINT	NOT NULL, FK → auth.Users	

Indexes:

- PK: `import_session_id`
- IX: `(org_id, status, created_at DESC)` — session listing
- IX: `(org_id, import_profile_id, created_at DESC)` — sessions by entity type
- IX: `(hangfire_job_id)` WHERE hangfire_job_id IS NOT NULL — job lookup

Session Status Lifecycle:

4.2.2 `import.ImportColumnMappings`

Column	Type	Constraints	Description
`import_column_mapping_id`	BIGINT	PK, IDENTITY	Auto-increment PK
`import_session_id`	BIGINT	NOT NULL, FK → import.ImportSessions	Parent session
`source_column_index`	INT	NOT NULL	0-based column index in the uploaded file
`source_column_name`	NVARCHAR(200)	NOT NULL	Header text from the uploaded file
`target_field_name`	VARCHAR(100)	NULL	Mapped SMS field name (NULL = column ignored/unmapped)
`target_field_id`	BIGINT	NULL, FK → import.ImportProfileFields	Resolved field reference
`is_auto_mapped`	BIT	NOT NULL, DEFAULT 0	True if auto-mapped by header name similarity
`confidence_score`	DECIMAL(5,2)	NULL	Auto-map confidence: 1.00 = exact match, 0.80+ = fuzzy match

Indexes:

- PK: `import_column_mapping_id`
- UQ: `(import_session_id, source_column_index)` — one mapping per source column
- IX: `(import_session_id, target_field_name)` — lookup by target field

4.2.3 `import.ImportRows`

Column	Type	Constraints	Description
`import_row_id`	BIGINT	PK, IDENTITY	Auto-increment PK
`import_session_id`	BIGINT	NOT NULL, FK → import.ImportSessions	Parent session
`row_number`	INT	NOT NULL	1-based row in the file (excluding header)
`status`	TINYINT	NOT NULL, DEFAULT 0	0=Pending, 1=Valid, 2=Invalid, 3=Imported, 4=Skipped, 5=Updated, 6=Error
`raw_data`	NVARCHAR(MAX)	NOT NULL	JSON object of the row's mapped field values: `{"sku":"PRD001","name":"Widget","category_code":"ELEC"}`
`created_entity_id`	BIGINT	NULL	ID of the created or updated SMS record
`error_details`	NVARCHAR(MAX)	NULL	JSON array of validation/import errors: `[{"field":"sku","code":"DUPLICATE","message":"SKU PRD001 already exists"}]`

Indexes:

- PK: `import_row_id`
- IX: `(import_session_id, status)` — filter by status (e.g., all invalid rows)
- IX: `(import_session_id, row_number)` — ordered row access

> **Note:** For very large imports (10k+ rows), `ImportRows` can grow significantly. Rows older than 90 days are eligible for archival via a Hangfire cleanup job.

---

5. C4 — External Reference Registry

5.1 Rationale

When importing data from a legacy system, each record has an ID in the source system (SAP material number, QuickBooks customer ID, legacy ERP product code). The external reference registry maps these source IDs to SMS entity IDs, enabling upsert behavior on subsequent imports and maintaining traceability back to the original system. This is equivalent to Odoo's `ir.model.data` table.

5.2 Entity

5.2.1 `import.ExternalReferences`

Column	Type	Constraints	Description
`external_reference_id`	BIGINT	PK, IDENTITY	Auto-increment PK
`org_id`	BIGINT	NOT NULL, FK → tenant.Organizations	Tenant
`entity_type`	TINYINT	NOT NULL	Same enum as ImportProfiles.entity_type
`entity_id`	BIGINT	NOT NULL	ID of the record in the target SMS table
`external_id`	NVARCHAR(200)	NOT NULL	Unique identifier from the source system
`source_system`	NVARCHAR(100)	NULL	Optional label: "SAP", "QuickBooks", "Legacy ERP", "Excel Migration"
`import_session_id`	BIGINT	NULL, FK → import.ImportSessions	Which import session created this reference
`created_at`	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Indexes:

- PK: `external_reference_id`
- UQ: `(org_id, entity_type, external_id)` — one external ID per entity type per org
- UQ: `(org_id, entity_type, entity_id)` — one reference per SMS record per entity type
- IX: `(org_id, entity_type, entity_id)` — reverse lookup: SMS ID → external ID
- IX: `(import_session_id)` — all references from one import

Usage Pattern:

```csharp
// During import — check if external_id already exists
public async Task<long?> ResolveExternalIdAsync(
    long orgId, ImportEntityType entityType, string externalId)
{
    var reference = await _db.ExternalReferences
        .FirstOrDefaultAsync(r =>
            r.OrgId == orgId &&
            r.EntityType == entityType &&
            r.ExternalId == externalId);

    return reference?.EntityId;
}

// After creating a new entity — register its external reference
public async Task RegisterExternalReferenceAsync(
    long orgId, ImportEntityType entityType, long entityId,
    string externalId, string? sourceSystem, long? sessionId)
{
    _db.ExternalReferences.Add(new ExternalReference
    {
        OrgId = orgId,
        EntityType = entityType,
        EntityId = entityId,
        ExternalId = externalId,
        SourceSystem = sourceSystem,
        ImportSessionId = sessionId
    });
    await _db.SaveChangesAsync();
}
```

6. C5 — Excel Template Engine

6.1 Rationale

Downloadable Excel templates reduce errors by giving users the exact column structure SMS expects, with data validation dropdowns for lookup fields, column-level instructions, and sample data rows. Templates are generated dynamically from the import profile's field definitions using ClosedXML.

6.2 NuGet Dependencies

Package	Version	License	Purpose
ClosedXML	0.102+	MIT	Excel (.xlsx) generation and reading
CsvHelper	31+	MS-PL / Apache 2.0	CSV parsing

6.3 Template Structure

Each generated template has 3 worksheets:

Sheet 1: "Data" (active sheet)

Row	Content
Row 1	Column headers (bold, frozen) — from `ImportProfileFields.display_name`
Row 2	Help text row (grey italic) — from `ImportProfileFields.description`
Row 3	Sample data row (light blue) — from `ImportProfileFields.sample_value`
Row 4+	Empty data rows for user to fill

- Columns have data validation applied:
  - **Lookup fields** with `lookup_dropdown = true`: dropdown list populated from current SMS data (e.g., all active category codes)
  - **Boolean fields**: dropdown with "Yes" / "No"
  - **Date fields**: Excel date format validation
  - **Decimal fields**: number validation with min/max and decimal places
- Required fields have red header background
- The `external_id` column (always column A) has a note: "Optional — Unique ID from your source system. Used for create-or-update on re-import."

Sheet 2: "Instructions"

Row	Content
1	"Import Instructions — {Entity Name}"
2	"Fill in the Data sheet. Required fields are highlighted in red."
3	"Lookup fields have dropdown lists. You can also type the code directly."
4	"Delete the sample row (row 3) before uploading."
5	"Maximum {max_rows} rows per file."
6	Field reference table: field name, required?, type, description

Sheet 3: "Lookups" (hidden)

Contains the reference data for dropdown validation lists (category codes, unit codes, warehouse codes, etc.). Auto-populated from current org data at template download time.

6.4 Template Generation Service

```csharp
```

```
public interface IImportTemplateService
{
    /// <summary>
    /// Generates an Excel template for the given import profile.
    /// Lookup dropdowns are populated from the org's current data.
    /// </summary>
    Task<byte[]> GenerateTemplateAsync(long orgId, long profileId);

    /// <summary>
    /// Generates a migration workbook — a single Excel file with one sheet
    /// per entity type in dependency order, each sheet structured as a template.
    /// </summary>
    Task<byte[]> GenerateMigrationWorkbookAsync(long orgId);
}
```

6.5 Migration Workbook

For full legacy migration, the `GenerateMigrationWorkbookAsync` method creates a single Excel file with sheets named by entity type in dependency order:

```
Sheet 1: "1 - Units"
Sheet 2: "2 - Categories"
Sheet 3: "3 - Products"
Sheet 4: "4 - Warehouses"
Sheet 5: "5 - Locations"
Sheet 6: "6 - Accounts"
Sheet 7: "7 - Tax Codes"
Sheet 8: "8 - Customers"
Sheet 9: "9 - Suppliers"
Sheet 10: "10 - Contacts"
Sheet 11: "11 - Employees"
Sheet 12: "12 - Price List Items"
Sheet 13: "13 - Opening Stock"
Sheet 14: "14 - Customer Balances"
Sheet 15: "15 - Supplier Balances"
Sheet 16: "16 - GL Balances"
```

Each sheet has the same structure as an individual template (headers, help row, sample row, validation). The user fills sheets in order and imports them one by one.

---

7. C6 — Column Mapping & Lookup Resolution

7.1 Column Auto-Mapping

When a file is uploaded, the system attempts to auto-map each source column to a target field:

Matching Algorithm (in priority order):

1. **Exact match** — source header matches `field_name` or `display_name` exactly (case-insensitive). Confidence = 1.00
2. **Normalized match** — after removing spaces, underscores, hyphens, and lowercasing. "Product Name" matches `product_name`. Confidence = 0.95

3. **Common alias match** — predefined aliases per field. "Item Code" → `sku`, "Part Number" → `sku`, "Material No" → `sku`. Confidence = 0.90
4. **Fuzzy match** — Levenshtein distance ≤ 2 on normalized names. "Prodct Name" → `product_name`. Confidence = 0.70–0.89

Auto-mappings with confidence < 0.70 are not applied. All auto-mappings are shown to the user for confirmation.

```csharp
public interface IColumnMappingService
{
    /// <summary>
    /// Auto-maps source columns to target fields.
    /// Returns proposed mappings for user review.
    /// </summary>
    Task<List<ColumnMappingProposal>> AutoMapColumnsAsync(
        long sessionId);

    /// <summary>
    /// Applies user-confirmed column mappings.
    /// </summary>
    Task ApplyMappingsAsync(
        long sessionId, List<ColumnMappingUpdate> mappings);

    /// <summary>
    /// Saves current mapping as a reusable preset.
    /// </summary>
    Task SaveMappingPresetAsync(
        long sessionId, string presetName);

    /// <summary>
    /// Loads a saved preset and applies its mappings to the session.
    /// </summary>
    Task LoadMappingPresetAsync(
        long sessionId, string presetName);
}
```

7.2 Saved Mapping Presets

When a user maps columns from a legacy system format (e.g., SAP export), they can save the mapping with a name like "SAP Material Export". Next time they import from the same source, they select the preset and all columns are auto-mapped according to the saved configuration.

Presets are stored as JSON on `ImportSessions.saved_mapping_name`. To load a preset, the system finds the most recent session with that `saved_mapping_name` and copies its `ImportColumnMappings`.

7.3 Lookup Resolution

For lookup fields (field_type = 5), the system resolves human-readable values to SMS entity IDs during validation:

```csharp
```

```
public interface ILookupResolverService
{
    /// <summary>
    /// Resolves a lookup value to an entity ID.
    /// E.g., category_code "ELECTRONICS" → category_id 42
    /// </summary>
    Task<long?> ResolveAsync(
        long orgId, ImportEntityType lookupEntityType,
        string lookupField, string value);

    /// <summary>
    /// Bulk-resolves all lookup values for a session.
    /// Caches resolved values for performance.
    /// </summary>
    Task<LookupCache> BuildLookupCacheAsync(
        long orgId, long sessionId);
}
```

Lookup resolution examples:

Source Column Value	Lookup Entity	Lookup Field	Resolves To
"ELECTRONICS"	ProductCategory	code	category_id = 42
"KG"	Unit	code	unit_id = 7
"MAIN-WH"	Warehouse	code	warehouse_id = 3
"4100"	Account	account_code	account_id = 156

If a lookup value cannot be resolved, the row is marked invalid with error code `LOOKUP_NOT_FOUND`.

---

8. C7 — Validation Engine

8.1 Rationale

Every row passes through a comprehensive validation pipeline before any database write. The validation is a dry-run — it reads the database to check referential integrity and uniqueness but writes nothing. After validation, the user sees exactly which rows will succeed and which will fail, with actionable error messages.

8.2 Validation Rules

Rule	Code	Description	Applies To
Required field	`REQUIRED`	Field is marked `is_required = true` and value is blank/null	All required fields
Data type	`INVALID_TYPE`	Value cannot be parsed as the declared type (integer, decimal, date, boolean)	Typed fields
Max length	`MAX_LENGTH`	Text value exceeds `max_length`	Text fields with max_length
Min/Max value	`OUT_OF_RANGE`	Numeric value outside `min_value` – `max_value` range	Numeric fields with bounds
Decimal precision	`INVALID_PRECISION`	More decimal places than `decimal_places` allows	Decimal fields
Regex pattern	`PATTERN_MISMATCH`	Value doesn't match `validation_regex`	Fields with regex
Email format	`INVALID_EMAIL`	Not a valid email address format	Email fields
Phone format	`INVALID_PHONE`	Not a valid phone number format (allows + and digits)	Phone fields
Lookup resolution	`LOOKUP_NOT_FOUND`	Lookup value doesn't match any existing record	Lookup fields
Duplicate in file	`DUPLICATE_IN_FILE`	Another row in the same file has the same key value	Key fields
Duplicate in DB	`DUPLICATE_IN_DB`	A record with this key already exists in SMS (and strategy = Error)	Key fields
Referential integrity	`MISSING_DEPENDENCY`	A dependency entity doesn't exist (e.g., product references non-existent category)	Lookup fields
Date range	`INVALID_DATE`	Date is out of reasonable range (e.g., future date for opening balance)	Date fields

8.3 Validation Service

csharp

```csharp
public interface IImportValidationService
{
    /// <summary>
    /// Runs validation on all rows in the session.
    /// Updates ImportRows with status and error_details.
    /// Updates ImportSessions with valid/invalid counts.
    /// </summary>
    Task<ValidationResult> ValidateSessionAsync(long sessionId);
}

public class ValidationResult
{
    public int TotalRows { get; set; }
    public int ValidRows { get; set; }
    public int InvalidRows { get; set; }
    public List<ValidationError> Errors { get; set; } = new();
}

public class ValidationError
{
    public int RowNumber { get; set; }
    public string FieldName { get; set; }
    public string ErrorCode { get; set; }   // e.g., "REQUIRED", "LOOKUP_NOT_FOUND"
    public string Message { get; set; }      // e.g., "Category 'ELEC123' not found"
    public string? SourceValue { get; set; } // The value that failed
}
```

8.4 Validation Processing

csharp

```
public async Task<ValidationResult> ValidateSessionAsync(long sessionId)
{
    var session = await GetSessionWithProfileAsync(sessionId);
    var fields = await GetProfileFieldsAsync(session.ImportProfileId);
    var mappings = await GetColumnMappingsAsync(sessionId);
    var lookupCache = await _lookupResolver.BuildLookupCacheAsync(
        session.OrgId, sessionId);

    // Load all rows
    var rows = await _db.ImportRows
        .Where(r => r.ImportSessionId == sessionId)
        .OrderBy(r => r.RowNumber)
        .ToListAsync();

    // Track in-file duplicates
    var keyTracker = new HashSet<string>();

    foreach (var row in rows)
    {
        var data = JsonSerializer.Deserialize<Dictionary<string, string?>>(row.RawData);
        var errors = new List<ValidationError>();

        foreach (var field in fields)
        {
            var value = data.GetValueOrDefault(field.FieldName);

            // 1. Required check
            if (field.IsRequired && string.IsNullOrWhiteSpace(value))
            {
                errors.Add(new ValidationError
                {
                    RowNumber = row.RowNumber,
                    FieldName = field.FieldName,
                    ErrorCode = "REQUIRED",
                    Message = $"'{field.DisplayName}' is required"
                });
                continue;
            }

            if (string.IsNullOrWhiteSpace(value)) continue;

            // 2. Type validation
            if (!ValidateFieldType(field, value, out var typeError))
                errors.Add(typeError);

            // 3. Length/range validation
            if (!ValidateConstraints(field, value, out var constraintError))
                errors.Add(constraintError);

            // 4. Lookup resolution
            if (field.FieldType == ImportFieldType.Lookup)
```

```
    {
        if (!lookupCache.TryResolve(field, value, out _))
        {
            errors.Add(new ValidationError
            {
                RowNumber = row.RowNumber,
                FieldName = field.FieldName,
                ErrorCode = "LOOKUP_NOT_FOUND",
                Message = $"'{value}' not found in {field.DisplayName}",
                SourceValue = value
            });
        }
    }

    // 5. In-file duplicate check (key fields)
    if (field.IsKey)
    {
        var keyValue = $"{field.FieldName}:{value}";
        if (!keyTracker.Add(keyValue))
        {
            errors.Add(new ValidationError
            {
                RowNumber = row.RowNumber,
                FieldName = field.FieldName,
                ErrorCode = "DUPLICATE_IN_FILE",
                Message = $"Duplicate {field.DisplayName} '{value}' in file"
            });
        }
    }
}

// 6. DB duplicate check (key fields combined)
if (errors.Count == 0)
{
    var isDuplicate = await CheckDatabaseDuplicateAsync(
        session, fields, data);
    if (isDuplicate && session.DuplicateStrategy == DuplicateStrategy.Error)
    {
        errors.Add(new ValidationError
        {
            RowNumber = row.RowNumber,
            FieldName = string.Join("+", fields.Where(f => f.IsKey).Select(f => f.FieldName)),
            ErrorCode = "DUPLICATE_IN_DB",
            Message = "Record already exists in SMS"
        });
    }
}

row.Status = errors.Count > 0
    ? ImportRowStatus.Invalid
    : ImportRowStatus.Valid;
```

```
            row.ErrorDetails = errors.Count > 0
                ? JsonSerializer.Serialize(errors)
                : null;
        }

        await _db.SaveChangesAsync();

        session.ValidRows = rows.Count(r => r.Status == ImportRowStatus.Valid);
        session.InvalidRows = rows.Count(r => r.Status == ImportRowStatus.Invalid);
        session.Status = ImportSessionStatus.Validated;
        session.ValidationCompletedAt = DateTime.UtcNow;
        await _db.SaveChangesAsync();

        return new ValidationResult { /* ... */ };
    }
```

---

9. C8 — Import Execution Pipeline

9.1 Rationale

After validation, the user triggers the actual import. The execution pipeline processes rows in configurable batches (default 100), creating or updating records in the target entity table. For files exceeding the background threshold (default 500 rows), execution runs as a Hangfire background job with real-time progress updates via SignalR.

9.2 Entity Import Handler Pattern

Each entity type has a dedicated handler implementing `IEntityImportHandler`. This keeps entity-specific logic (column-to-property mapping, related entity creation, special business rules) isolated and testable.

csharp

```
public interface IEntityImportHandler
{
    ImportEntityType EntityType { get; }

    /// <summary>
    /// Creates a new entity from the imported row data.
    /// Returns the created entity's ID.
    /// </summary>
    Task<long> CreateAsync(
        long orgId, long userId,
        Dictionary<string, string?> rowData,
        LookupCache lookupCache);

    /// <summary>
    /// Updates an existing entity with the imported row data.
    /// </summary>
    Task UpdateAsync(
        long orgId, long entityId, long userId,
        Dictionary<string, string?> rowData,
        LookupCache lookupCache);
}
```

Registered Handlers:

Entity Type	Handler Class	Target
Customer	`CustomerImportHandler`	customers.Customers
Supplier	`SupplierImportHandler`	procurement.Suppliers
Product	`ProductImportHandler`	inventory.Products
ProductCategory	`ProductCategoryImportHandler`	inventory.ProductCategories
UnitOfMeasure	`UnitImportHandler`	inventory.Units
Warehouse	`WarehouseImportHandler`	inventory.Warehouses
WarehouseLocation	`WarehouseLocationImportHandler`	inventory.WarehouseLocations
Account	`AccountImportHandler`	finance.Accounts
TaxCode	`TaxCodeImportHandler`	finance.TaxCodes
Contact	`ContactImportHandler`	core.Contacts
PriceListItem	`PriceListItemImportHandler`	inventory.PriceListItems
Employee	`EmployeeImportHandler`	hr.Employees
OpeningStock	`OpeningStockImportHandler`	(special — see C9)
CustomerOpeningBalance	`CustomerBalanceImportHandler`	(special — see C9)
SupplierOpeningBalance	`SupplierBalanceImportHandler`	(special — see C9)
GLOpeningBalance	`GLBalanceImportHandler`	(special — see C9)

Handler Registration (DI):

csharp

```csharp
services.AddScoped<IEntityImportHandler, CustomerImportHandler>();
services.AddScoped<IEntityImportHandler, SupplierImportHandler>();
services.AddScoped<IEntityImportHandler, ProductImportHandler>();
// ... all 16 handlers

services.AddScoped<IEntityImportHandlerFactory, EntityImportHandlerFactory>();
```

9.3 Import Execution Service

csharp

```csharp
public interface IImportExecutionService
{
    /// <summary>
    /// Executes the import for a validated session.
    /// Processes rows in batches. Returns immediately for background jobs.
    /// </summary>
    Task<ImportExecutionResult> ExecuteAsync(long sessionId);

    /// <summary>
    /// Gets real-time progress for a running import.
    /// </summary>
    Task<ImportProgress> GetProgressAsync(long sessionId);

    /// <summary>
    /// Cancels a running background import.
    /// </summary>
    Task CancelAsync(long sessionId);
}

public class ImportExecutionResult
{
    public long SessionId { get; set; }
    public bool IsBackground { get; set; }
    public string? HangfireJobId { get; set; }
    public int ImportedRows { get; set; }
    public int UpdatedRows { get; set; }
    public int SkippedRows { get; set; }
    public int ErrorRows { get; set; }
}
```

9.4 Batch Processing Logic

csharp

```
public async Task ProcessBatchAsync(long sessionId, int batchSize = 100)
{
    var session = await GetSessionAsync(sessionId);
    var handler = _handlerFactory.GetHandler(session.Profile.EntityType);
    var lookupCache = await _lookupResolver.BuildLookupCacheAsync(
        session.OrgId, sessionId);

    var validRows = await _db.ImportRows
        .Where(r => r.ImportSessionId == sessionId && r.Status == ImportRowStatus.Valid)
        .OrderBy(r => r.RowNumber)
        .ToListAsync();

    foreach (var batch in validRows.Chunk(batchSize))
    {
        using var transaction = await _db.Database.BeginTransactionAsync();
        try
        {
            foreach (var row in batch)
            {
                var data = JsonSerializer.Deserialize<Dictionary<string, string?>>(
                    row.RawData);
                var externalId = data.GetValueOrDefault("external_id");

                // Check for existing record (by external_id or match fields)
                long? existingId = null;
                if (!string.IsNullOrEmpty(externalId))
                {
                    existingId = await _externalRefService.ResolveExternalIdAsync(
                        session.OrgId, session.Profile.EntityType, externalId);
                }
                if (existingId == null)
                {
                    existingId = await CheckMatchFieldsAsync(
                        session, handler, data, lookupCache);
                }

                if (existingId != null)
                {
                    // Record exists — apply duplicate strategy
                    switch (session.DuplicateStrategy)
                    {
                        case DuplicateStrategy.Skip:
                            row.Status = ImportRowStatus.Skipped;
                            session.SkippedRows++;
                            break;

                        case DuplicateStrategy.Update:
                            await handler.UpdateAsync(
                                session.OrgId, existingId.Value,
                                session.CreatedBy, data, lookupCache);
                            row.Status = ImportRowStatus.Updated;
```

```
                    row.CreatedEntityId = existingId.Value;
                    session.UpdatedRows++;
                    break;

                case DuplicateStrategy.CreateNew:
                    var newId = await handler.CreateAsync(
                        session.OrgId, session.CreatedBy,
                        data, lookupCache);
                    row.Status = ImportRowStatus.Imported;
                    row.CreatedEntityId = newId;
                    session.ImportedRows++;
                    break;
            }
        }
        else
        {
            // New record — create
            var entityId = await handler.CreateAsync(
                session.OrgId, session.CreatedBy,
                data, lookupCache);
            row.Status = ImportRowStatus.Imported;
            row.CreatedEntityId = entityId;
            session.ImportedRows++;

            // Register external reference if provided
            if (!string.IsNullOrEmpty(externalId))
            {
                await _externalRefService.RegisterExternalReferenceAsync(
                    session.OrgId, session.Profile.EntityType,
                    entityId, externalId, null, sessionId);
            }
        }
    }

    await _db.SaveChangesAsync();
    await transaction.CommitAsync();

    // Send progress update via SignalR
    await _hubContext.Clients.User(session.CreatedBy.ToString())
        .SendAsync("ImportProgress", new
        {
            sessionId,
            processed = session.ImportedRows + session.UpdatedRows +
                        session.SkippedRows + session.ErrorRows,
            total = session.TotalRows
        });
}
catch (Exception ex)
{
    await transaction.RollbackAsync();
    foreach (var row in batch.Where(r => r.Status == ImportRowStatus.Valid))
```

```
            {
                row.Status = ImportRowStatus.Error;
                row.ErrorDetails = JsonSerializer.Serialize(new[]
                {
                    new { field = "", code = "IMPORT_ERROR", message = ex.Message }
                });
                session.ErrorRows++;
            }
            await _db.SaveChangesAsync();
        }
    }
}
```

9.5 Hangfire Background Job

csharp

```
public class ImportExecutionJob
{
    private readonly IImportExecutionService _importService;

    public ImportExecutionJob(IImportExecutionService importService)
    {
        _importService = importService;
    }

    [AutomaticRetry(Attempts = 0)] // No auto-retry on import failures
    [Queue("import")]
    public async Task ExecuteAsync(long sessionId, CancellationToken cancellationToken)
    {
        await _importService.ProcessBatchAsync(sessionId);
    }
}
```

Threshold for background processing:

csharp

```
public async Task<ImportExecutionResult> ExecuteAsync(long sessionId)
{
    var session = await GetSessionAsync(sessionId);
    var backgroundThreshold = 500; // configurable per org

    if (session.ValidRows > backgroundThreshold)
    {
        // Queue as Hangfire job
        var jobId = BackgroundJob.Enqueue<ImportExecutionJob>(
            job => job.ExecuteAsync(sessionId, CancellationToken.None));

        session.HangfireJobId = jobId;
        session.Status = ImportSessionStatus.Importing;
        await _db.SaveChangesAsync();

        return new ImportExecutionResult
        {
            SessionId = sessionId,
            IsBackground = true,
            HangfireJobId = jobId
        };
    }
    else
    {
        // Process synchronously
        session.Status = ImportSessionStatus.Importing;
        session.ImportStartedAt = DateTime.UtcNow;
        await _db.SaveChangesAsync();

        await ProcessBatchAsync(sessionId);

        session.Status = session.ErrorRows > 0
            ? ImportSessionStatus.CompletedWithErrors
            : ImportSessionStatus.Completed;
        session.ImportCompletedAt = DateTime.UtcNow;
        await _db.SaveChangesAsync();

        return new ImportExecutionResult { /* counts */ };
    }
}
```

---

10. C9 — Opening Balance Import

10.1 Rationale

Opening balances are not simple entity inserts. They generate journal entries and stock movements to establish the initial state of the SMS system. Each opening balance type has a special handler that creates the correct accounting entries alongside the data records.

10.2 Opening Stock Import

Template columns: `product_sku`, `warehouse_code`, `location_code (optional)`, `quantity`, `unit_cost`, `batch_number (optional)`, `expiry_date (optional)`

Handler logic:

csharp

```
public class OpeningStockImportHandler : IEntityImportHandler
{
    public ImportEntityType EntityType => ImportEntityType.OpeningStock;

    public async Task<long> CreateAsync(
        long orgId, long userId,
        Dictionary<string, string?> rowData, LookupCache lookupCache)
    {
        var productId = lookupCache.ResolveRequired("product_sku", rowData["product_sku"]);
        var warehouseId = lookupCache.ResolveRequired("warehouse_code", rowData["warehouse_code"]);
        var quantity = decimal.Parse(rowData["quantity"]!);
        var unitCost = decimal.Parse(rowData["unit_cost"]!);

        // 1. Create stock movement (type = OPENING_BALANCE, direction = IN)
        var movement = new StockMovement
        {
            OrgId = orgId,
            ProductId = productId,
            WarehouseId = warehouseId,
            MovementType = StockMovementType.OpeningBalance,
            Direction = StockDirection.In,
            Quantity = quantity,
            UnitCost = unitCost,
            TotalCost = quantity * unitCost,
            Reference = "OPENING-BALANCE",
            CreatedBy = userId
        };
        _db.StockMovements.Add(movement);

        // 2. Update stock level
        var stockLevel = await _db.StockLevels
            .FirstOrDefaultAsync(s =>
                s.OrgId == orgId &&
                s.ProductId == productId &&
                s.WarehouseId == warehouseId);

        if (stockLevel != null)
        {
            stockLevel.QuantityOnHand += quantity;
            stockLevel.TotalCost += quantity * unitCost;
        }
        else
        {
            _db.StockLevels.Add(new StockLevel
            {
                OrgId = orgId,
                ProductId = productId,
                WarehouseId = warehouseId,
                QuantityOnHand = quantity,
                TotalCost = quantity * unitCost
            });
```

```
        }

        // 3. Generate journal entry
        //    DR: Inventory Asset (quantity × unit_cost)
        //    CR: Opening Balance Equity (quantity × unit_cost)
        await _journalService.CreateOpeningStockEntryAsync(
            orgId, productId, warehouseId,
            quantity * unitCost, userId);

        await _db.SaveChangesAsync();
        return movement.StockMovementId;
    }

    public Task UpdateAsync(/*...*/) =>
        throw new NotSupportedException("Opening balances cannot be updated via import. Reverse and re-import.");
}
```

10.3 Customer Opening Balance Import

Template columns: `customer_code`, `invoice_reference (optional)`, `invoice_date`, `due_date`, `amount`, `currency_code (optional)`

Handler logic:

- Creates an opening balance journal entry per row
- DR: Accounts Receivable (customer sub-ledger)
- CR: Opening Balance Equity
- Sets up the customer's outstanding balance in the AR sub-ledger

10.4 Supplier Opening Balance Import

Template columns: `supplier_code`, `bill_reference (optional)`, `bill_date`, `due_date`, `amount`, `currency_code (optional)`

Handler logic:

- DR: Opening Balance Equity
- CR: Accounts Payable (supplier sub-ledger)

10.5 GL Opening Balance Import

Template columns: `account_code`, `debit_amount`, `credit_amount`, `description (optional)`

Handler logic:

- Creates a single opening balance journal entry with all rows as lines
- Total debits must equal total credits (validated before import)
- The journal entry date is the organization's fiscal year start date

Special validation rule:

csharp

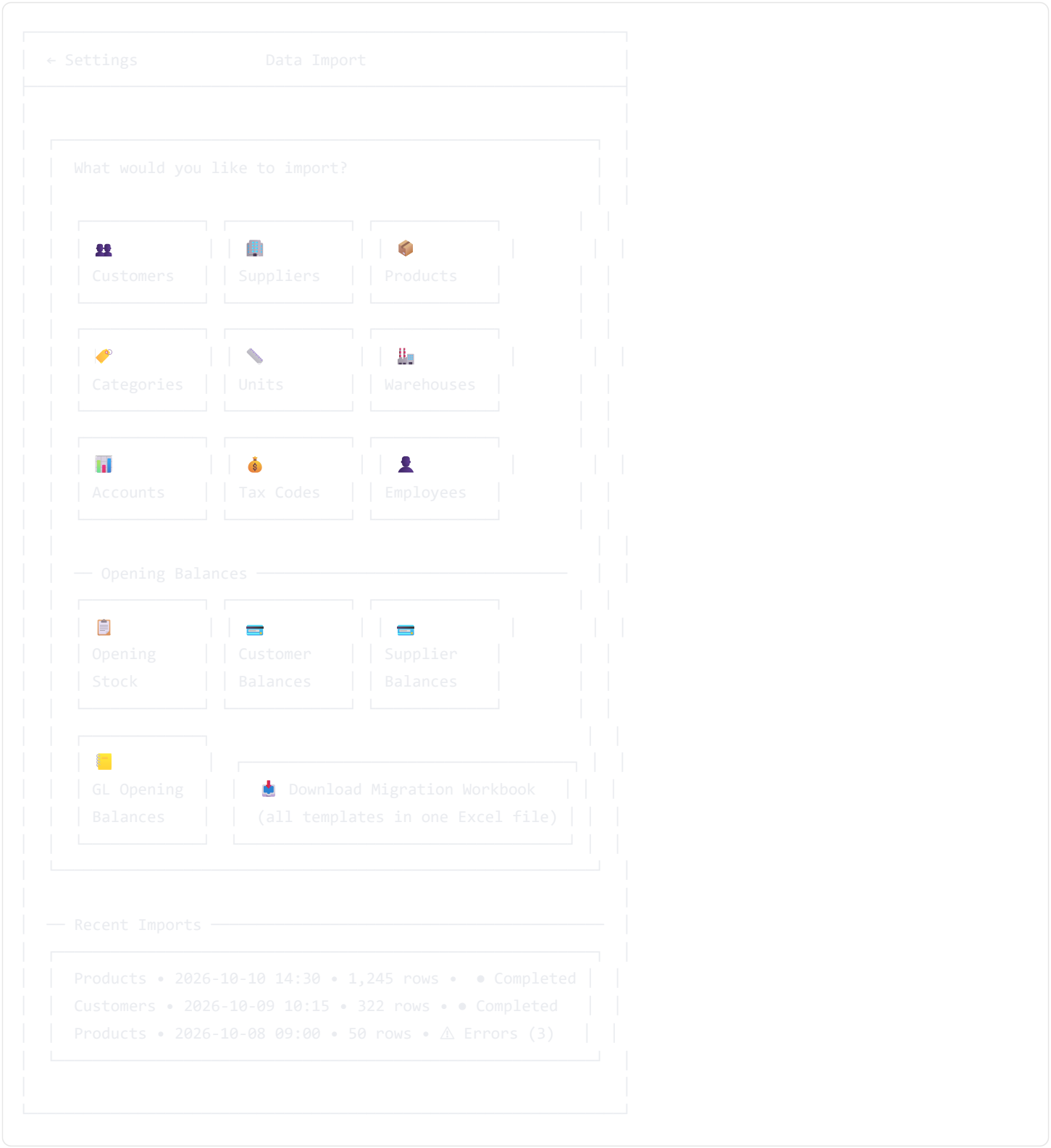
```
// GL Opening Balance requires total DR = total CR
var totalDebit = rows.Sum(r => decimal.Parse(r["debit_amount"] ?? "0"));
var totalCredit = rows.Sum(r => decimal.Parse(r["credit_amount"] ?? "0"));
if (totalDebit != totalCredit)
{
    throw new BusinessException(
        $"Opening balances are unbalanced. " +
        $"Total Debit: {totalDebit:N2}, Total Credit: {totalCredit:N2}, " +
        $"Difference: {Math.Abs(totalDebit - totalCredit):N2}");
}
```

---

11. Import UI Screens

11.1 Import Dashboard

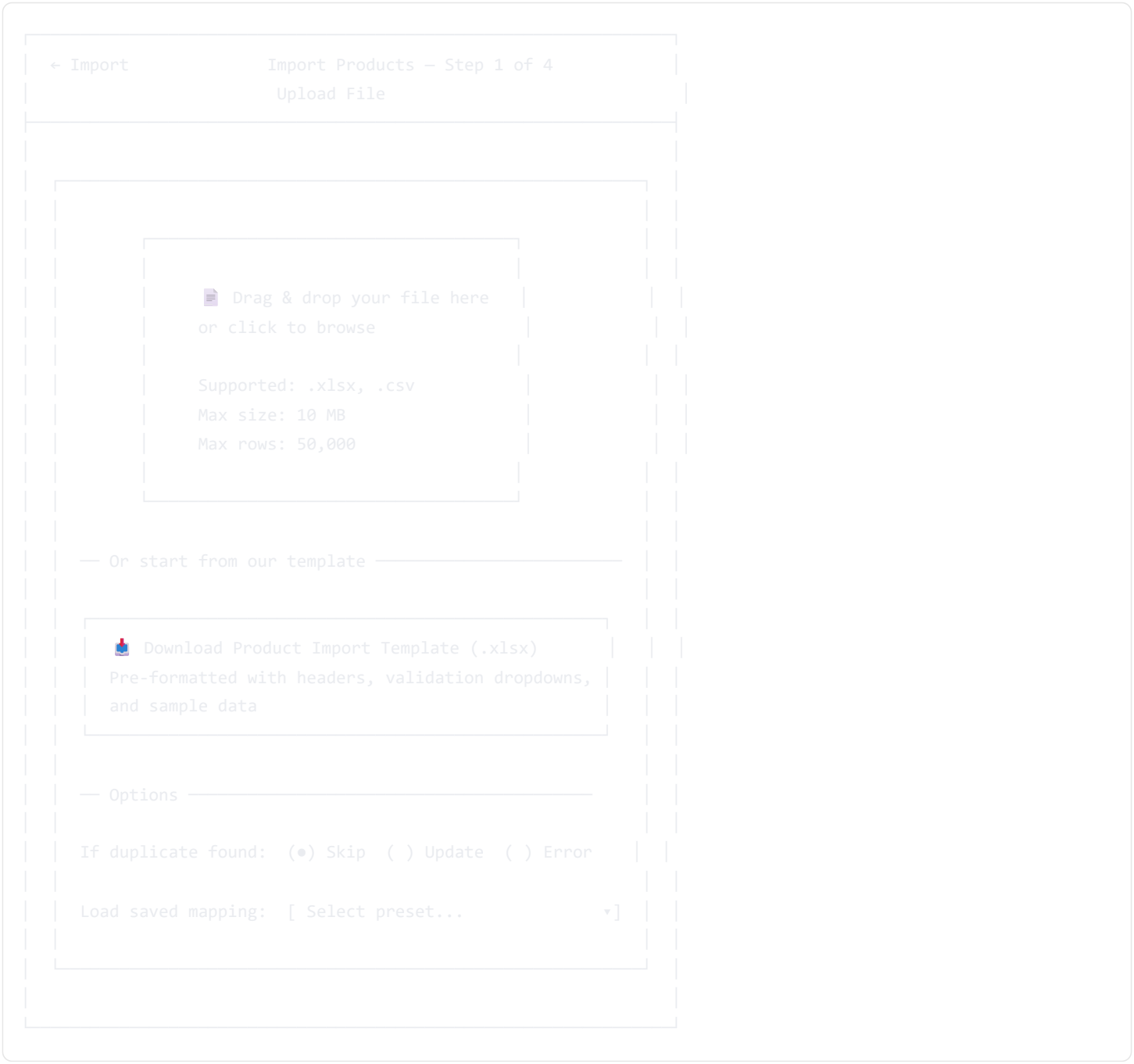
Route: `/settings/import`



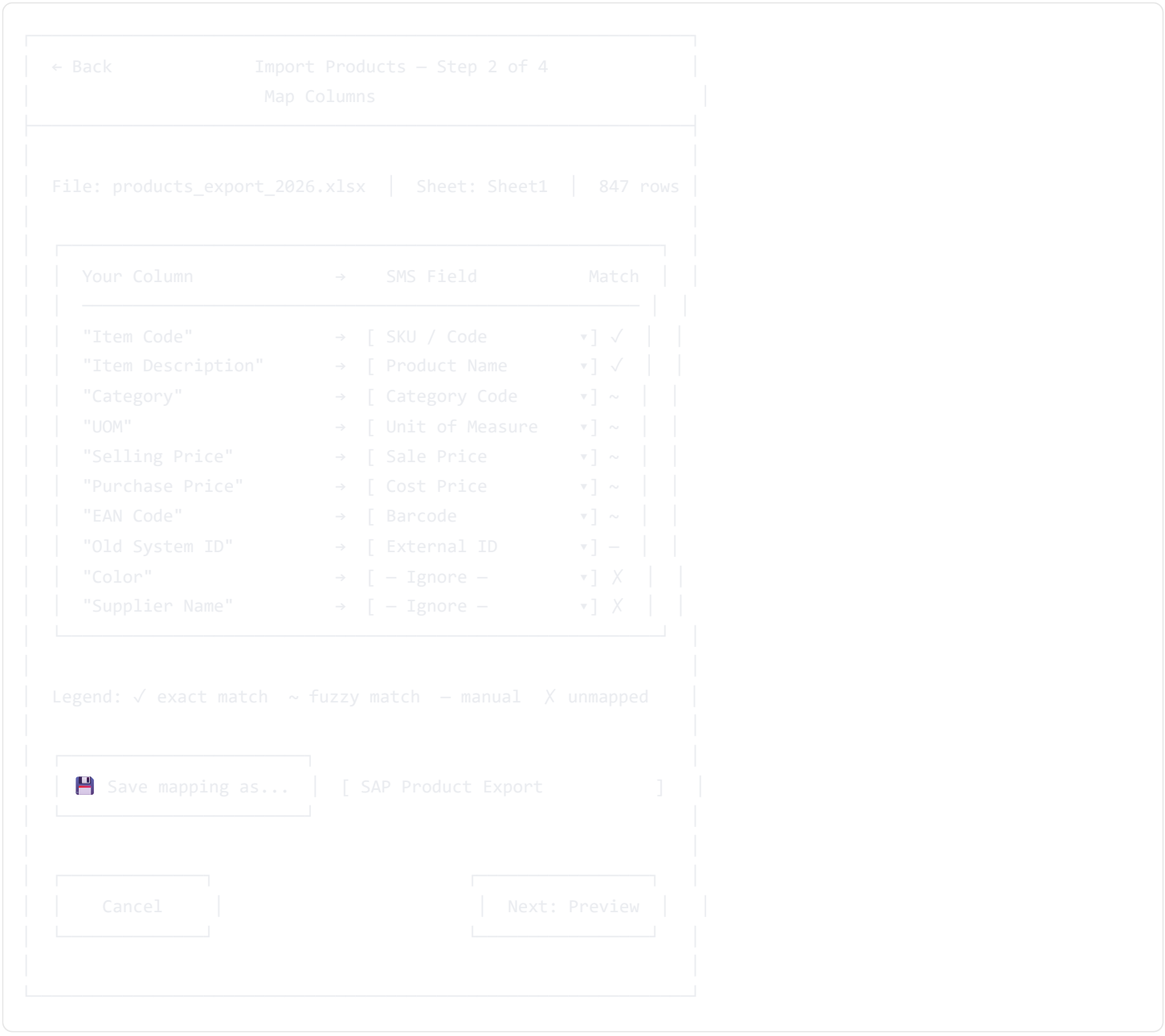
11.2 Upload & Mapping Wizard

Route: `/settings/import/{entityType}/upload`

Step 1 — Upload File:



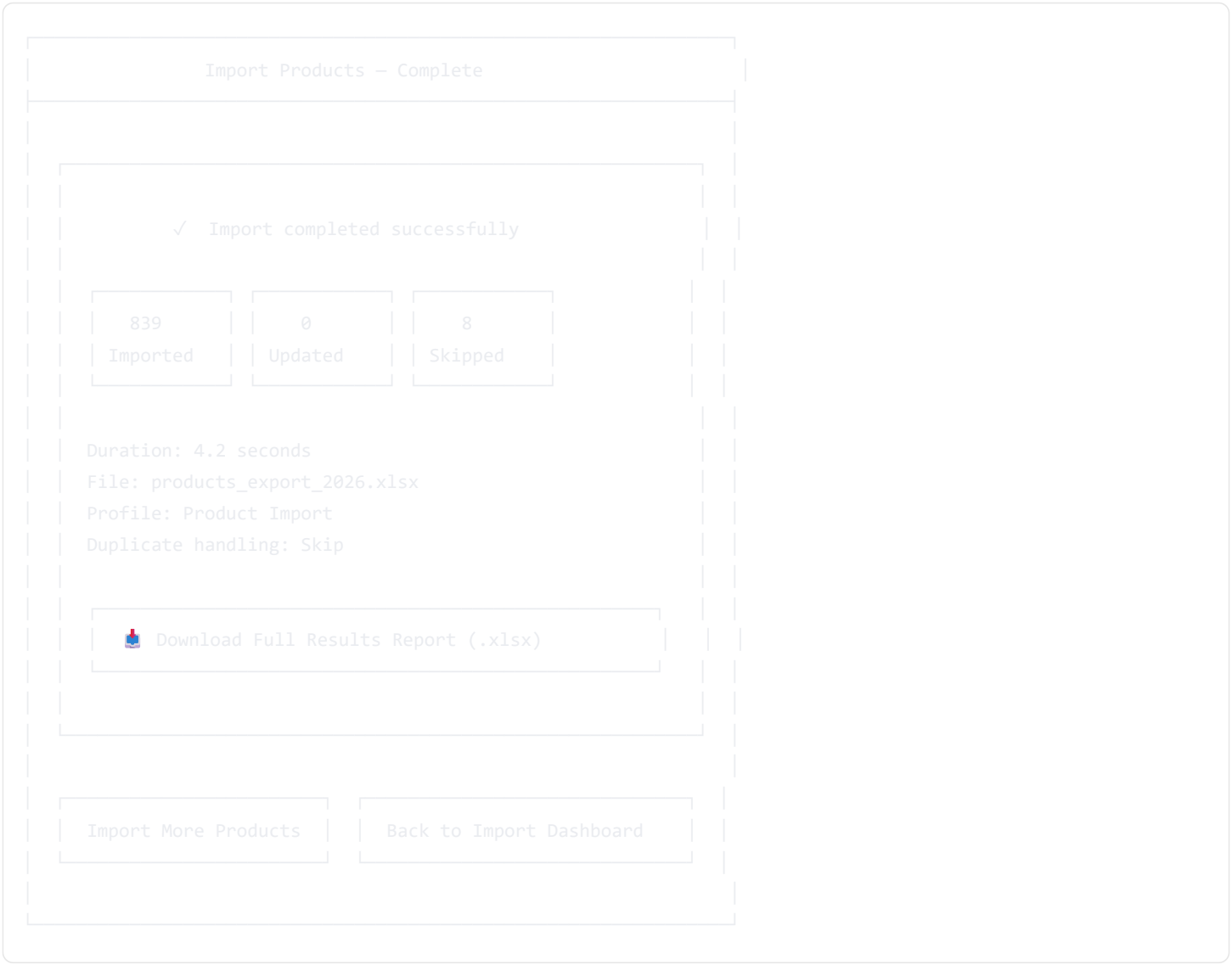
Step 2 — Column Mapping:



Step 3 — Preview & Validate:



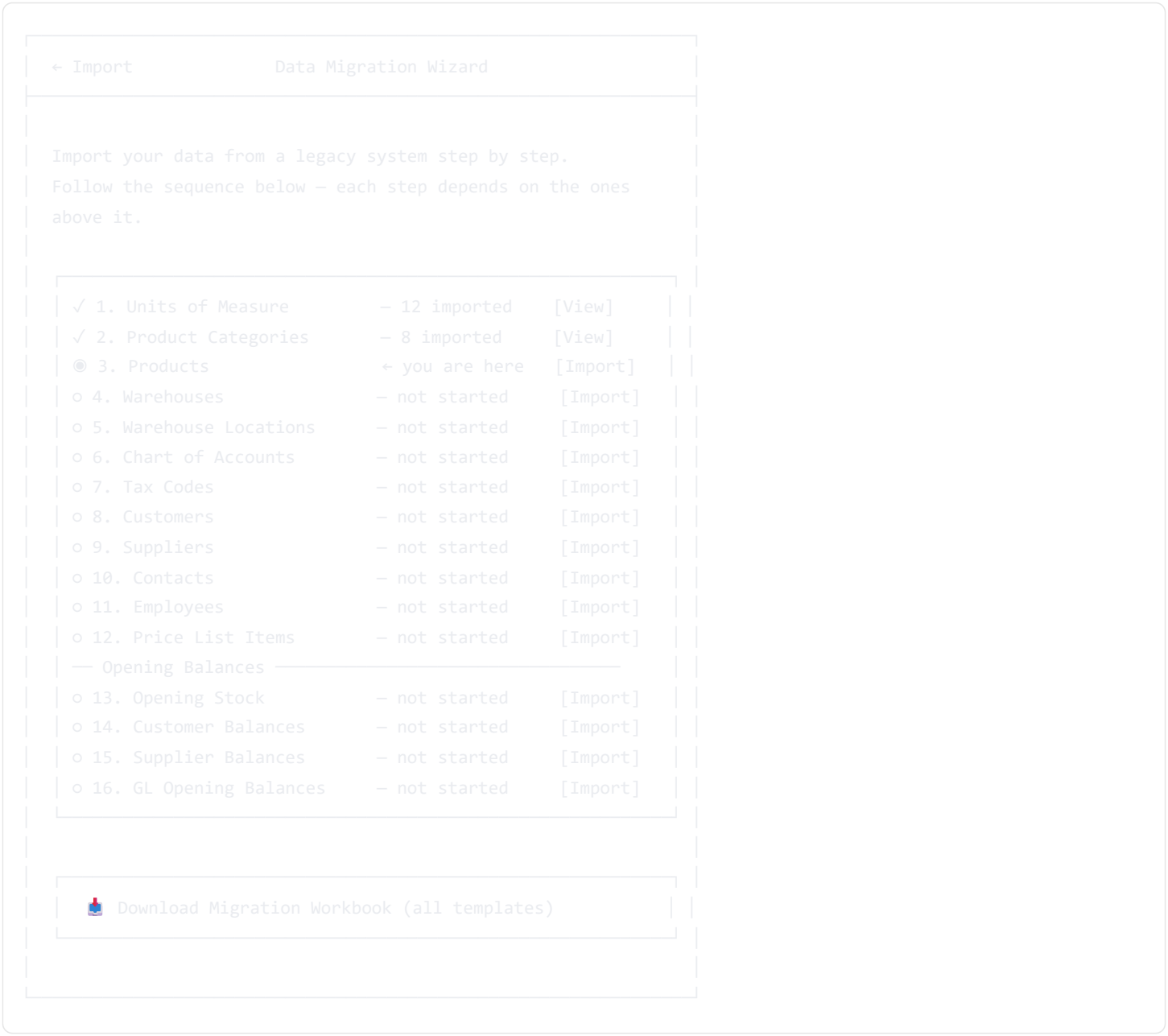
Step 4 — Results:



11.3 Migration Wizard

Route: `/settings/import/migration`

A guided walkthrough for first-time setup that shows all entity types in dependency order with a checklist:



---

12. Database Migrations

M040.1 — Create Import Schema

sql

```sql
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'import')
    EXEC('CREATE SCHEMA [import]');
```

M040.2 — Create ImportProfiles Table

sql

```sql
CREATE TABLE [import].[ImportProfiles] (
    [import_profile_id]   BIGINT          IDENTITY(1,1) NOT NULL,
    [org_id]              BIGINT          NULL,
    [entity_type]         TINYINT         NOT NULL,
    [name]                NVARCHAR(200)   NOT NULL,
    [description]         NVARCHAR(500)   NULL,
    [target_schema]       VARCHAR(50)     NOT NULL,
    [target_table]        VARCHAR(100)    NOT NULL,
    [match_fields]        NVARCHAR(300)   NOT NULL,
    [duplicate_strategy]  TINYINT         NOT NULL   DEFAULT 0,
    [import_order]        INT             NOT NULL   DEFAULT 0,
    [max_rows_per_file]   INT             NOT NULL   DEFAULT 50000,
    [max_file_size_mb]    INT             NOT NULL   DEFAULT 10,
    [is_opening_balance]  BIT             NOT NULL   DEFAULT 0,
    [is_system]           BIT             NOT NULL   DEFAULT 1,
    [is_active]           BIT             NOT NULL   DEFAULT 1,
    [created_at]          DATETIME2(2)    NOT NULL   DEFAULT SYSUTCDATETIME(),
    [created_by]          BIGINT          NULL,
    [modified_at]         DATETIME2(2)    NOT NULL   DEFAULT SYSUTCDATETIME(),
    [modified_by]         BIGINT          NULL,

    CONSTRAINT [PK_ImportProfiles] PRIMARY KEY ([import_profile_id]),
    CONSTRAINT [FK_ImportProfiles_Org] FOREIGN KEY ([org_id])
        REFERENCES [tenant].[Organizations] ([organization_id]),
    CONSTRAINT [FK_ImportProfiles_CreatedBy] FOREIGN KEY ([created_by])
        REFERENCES [auth].[Users] ([user_id]),
    CONSTRAINT [FK_ImportProfiles_ModifiedBy] FOREIGN KEY ([modified_by])
        REFERENCES [auth].[Users] ([user_id])
);

CREATE UNIQUE INDEX [UQ_ImportProfiles_OrgEntityType]
    ON [import].[ImportProfiles] ([org_id], [entity_type])
    WHERE [org_id] IS NOT NULL;

CREATE INDEX [IX_ImportProfiles_EntityType]
    ON [import].[ImportProfiles] ([entity_type], [is_active]);

CREATE INDEX [IX_ImportProfiles_OrgOrder]
    ON [import].[ImportProfiles] ([org_id], [is_active], [import_order]);
```

M040.3 — Create ImportProfileFields Table

sql

```sql
CREATE TABLE [import].[ImportProfileFields] (
    [import_profile_field_id]  BIGINT          IDENTITY(1,1) NOT NULL,
    [import_profile_id]        BIGINT          NOT NULL,
    [field_name]               VARCHAR(100)    NOT NULL,
    [display_name]             NVARCHAR(200)   NOT NULL,
    [description]              NVARCHAR(500)   NULL,
    [field_type]               TINYINT         NOT NULL,
    [is_required]              BIT             NOT NULL   DEFAULT 0,
    [is_key]                   BIT             NOT NULL   DEFAULT 0,
    [is_template_column]       BIT             NOT NULL   DEFAULT 1,
    [max_length]               INT             NULL,
    [min_value]                DECIMAL(18,4)   NULL,
    [max_value]                DECIMAL(18,4)   NULL,
    [decimal_places]           TINYINT         NULL,
    [validation_regex]         NVARCHAR(500)   NULL,
    [default_value]            NVARCHAR(500)   NULL,
    [lookup_entity_type]       TINYINT         NULL,
    [lookup_match_field]       VARCHAR(100)    NULL,
    [lookup_dropdown]          BIT             NOT NULL   DEFAULT 0,
    [sort_order]               INT             NOT NULL   DEFAULT 0,
    [sample_value]             NVARCHAR(200)   NULL,

    CONSTRAINT [PK_ImportProfileFields] PRIMARY KEY ([import_profile_field_id]),
    CONSTRAINT [FK_ImportProfileFields_Profile] FOREIGN KEY ([import_profile_id])
        REFERENCES [import].[ImportProfiles] ([import_profile_id])
        ON DELETE CASCADE
);

CREATE UNIQUE INDEX [UQ_ImportProfileFields_FieldName]
    ON [import].[ImportProfileFields] ([import_profile_id], [field_name]);

CREATE INDEX [IX_ImportProfileFields_Order]
    ON [import].[ImportProfileFields] ([import_profile_id], [sort_order]);
```

M040.4 — Create ImportSessions Table

sql

```sql
CREATE TABLE [import].[ImportSessions] (
    [import_session_id]         BIGINT          IDENTITY(1,1) NOT NULL,
    [org_id]                    BIGINT          NOT NULL,
    [import_profile_id]         BIGINT          NOT NULL,
    [file_name]                 NVARCHAR(500)   NOT NULL,
    [file_size_bytes]           BIGINT          NOT NULL,
    [file_type]                 TINYINT         NOT NULL,
    [sheet_name]                NVARCHAR(200)   NULL,
    [duplicate_strategy]        TINYINT         NOT NULL   DEFAULT 0,
    [total_rows]                INT             NOT NULL   DEFAULT 0,
    [valid_rows]                INT             NOT NULL   DEFAULT 0,
    [invalid_rows]              INT             NOT NULL   DEFAULT 0,
    [imported_rows]             INT             NOT NULL   DEFAULT 0,
    [updated_rows]              INT             NOT NULL   DEFAULT 0,
    [skipped_rows]              INT             NOT NULL   DEFAULT 0,
    [error_rows]                INT             NOT NULL   DEFAULT 0,
    [status]                    TINYINT         NOT NULL   DEFAULT 0,
    [validation_started_at]     DATETIME2(2)    NULL,
    [validation_completed_at]   DATETIME2(2)    NULL,
    [import_started_at]         DATETIME2(2)    NULL,
    [import_completed_at]       DATETIME2(2)    NULL,
    [error_message]             NVARCHAR(MAX)   NULL,
    [hangfire_job_id]           VARCHAR(200)    NULL,
    [saved_mapping_name]        NVARCHAR(200)   NULL,
    [notes]                     NVARCHAR(1000)  NULL,
    [created_at]                DATETIME2(2)    NOT NULL   DEFAULT SYSUTCDATETIME(),
    [created_by]                BIGINT          NOT NULL,

    CONSTRAINT [PK_ImportSessions] PRIMARY KEY ([import_session_id]),
    CONSTRAINT [FK_ImportSessions_Org] FOREIGN KEY ([org_id])
        REFERENCES [tenant].[Organizations] ([organization_id]),
    CONSTRAINT [FK_ImportSessions_Profile] FOREIGN KEY ([import_profile_id])
        REFERENCES [import].[ImportProfiles] ([import_profile_id]),
    CONSTRAINT [FK_ImportSessions_CreatedBy] FOREIGN KEY ([created_by])
        REFERENCES [auth].[Users] ([user_id])
);

CREATE INDEX [IX_ImportSessions_OrgStatus]
    ON [import].[ImportSessions] ([org_id], [status], [created_at] DESC);

CREATE INDEX [IX_ImportSessions_OrgProfile]
    ON [import].[ImportSessions] ([org_id], [import_profile_id], [created_at] DESC);

CREATE INDEX [IX_ImportSessions_Hangfire]
    ON [import].[ImportSessions] ([hangfire_job_id])
    WHERE [hangfire_job_id] IS NOT NULL;
```

M040.5 — Create ImportColumnMappings Table

sql

```sql
CREATE TABLE [import].[ImportColumnMappings] (
    [import_column_mapping_id]  BIGINT          IDENTITY(1,1) NOT NULL,
    [import_session_id]         BIGINT          NOT NULL,
    [source_column_index]       INT             NOT NULL,
    [source_column_name]        NVARCHAR(200)   NOT NULL,
    [target_field_name]         VARCHAR(100)    NULL,
    [target_field_id]           BIGINT          NULL,
    [is_auto_mapped]            BIT             NOT NULL    DEFAULT 0,
    [confidence_score]          DECIMAL(5,2)    NULL,

    CONSTRAINT [PK_ImportColumnMappings] PRIMARY KEY ([import_column_mapping_id]),
    CONSTRAINT [FK_ImportColumnMappings_Session] FOREIGN KEY ([import_session_id])
        REFERENCES [import].[ImportSessions] ([import_session_id])
        ON DELETE CASCADE,
    CONSTRAINT [FK_ImportColumnMappings_Field] FOREIGN KEY ([target_field_id])
        REFERENCES [import].[ImportProfileFields] ([import_profile_field_id])
);

CREATE UNIQUE INDEX [UQ_ImportColumnMappings_SourceCol]
    ON [import].[ImportColumnMappings] ([import_session_id], [source_column_index]);

CREATE INDEX [IX_ImportColumnMappings_TargetField]
    ON [import].[ImportColumnMappings] ([import_session_id], [target_field_name]);
```

M040.6 — Create ImportRows Table

sql

```sql
CREATE TABLE [import].[ImportRows] (
    [import_row_id]       BIGINT          IDENTITY(1,1) NOT NULL,
    [import_session_id]   BIGINT          NOT NULL,
    [row_number]          INT             NOT NULL,
    [status]              TINYINT         NOT NULL    DEFAULT 0,
    [raw_data]            NVARCHAR(MAX)   NOT NULL,
    [created_entity_id]   BIGINT          NULL,
    [error_details]       NVARCHAR(MAX)   NULL,

    CONSTRAINT [PK_ImportRows] PRIMARY KEY ([import_row_id]),
    CONSTRAINT [FK_ImportRows_Session] FOREIGN KEY ([import_session_id])
        REFERENCES [import].[ImportSessions] ([import_session_id])
        ON DELETE CASCADE
);

CREATE INDEX [IX_ImportRows_Status]
    ON [import].[ImportRows] ([import_session_id], [status]);

CREATE INDEX [IX_ImportRows_RowNumber]
    ON [import].[ImportRows] ([import_session_id], [row_number]);
```

M040.7 — Create ExternalReferences Table

```sql
CREATE TABLE [import].[ExternalReferences] (
    [external_reference_id]  BIGINT          IDENTITY(1,1) NOT NULL,
    [org_id]                 BIGINT          NOT NULL,
    [entity_type]            TINYINT         NOT NULL,
    [entity_id]              BIGINT          NOT NULL,
    [external_id]            NVARCHAR(200)   NOT NULL,
    [source_system]          NVARCHAR(100)   NULL,
    [import_session_id]      BIGINT          NULL,
    [created_at]             DATETIME2(2)    NOT NULL   DEFAULT SYSUTCDATETIME(),

    CONSTRAINT [PK_ExternalReferences] PRIMARY KEY ([external_reference_id]),
    CONSTRAINT [FK_ExternalReferences_Org] FOREIGN KEY ([org_id])
        REFERENCES [tenant].[Organizations] ([organization_id]),
    CONSTRAINT [FK_ExternalReferences_Session] FOREIGN KEY ([import_session_id])
        REFERENCES [import].[ImportSessions] ([import_session_id])
);

CREATE UNIQUE INDEX [UQ_ExternalReferences_ExternalId]
    ON [import].[ExternalReferences] ([org_id], [entity_type], [external_id]);

CREATE UNIQUE INDEX [UQ_ExternalReferences_EntityId]
    ON [import].[ExternalReferences] ([org_id], [entity_type], [entity_id]);

CREATE INDEX [IX_ExternalReferences_Session]
    ON [import].[ExternalReferences] ([import_session_id]);
```

M040.8 — Seed System Import Profiles

```sql
```

```
SET IDENTITY_INSERT [import].[ImportProfiles] ON;

INSERT INTO [import].[ImportProfiles]
    ([import_profile_id], [org_id], [entity_type], [name], [description],
     [target_schema], [target_table], [match_fields], [duplicate_strategy],
     [import_order], [is_opening_balance], [is_system], [is_active], [created_at])
VALUES
    (1, NULL, 0, 'Customer Import', 'Import customers with contact details, credit limits, and payment terms', 'c
    (2, NULL, 1, 'Supplier Import', 'Import suppliers with contact details and payment terms', 'procurement', 'Su
    (3, NULL, 2, 'Product Import', 'Import products with category, unit, pricing, and stock settings', 'inventory
    (4, NULL, 3, 'Product Category Import', 'Import product categories with hierarchy', 'inventory', 'ProductCate
    (5, NULL, 4, 'Unit of Measure Import', 'Import units of measure', 'inventory', 'Units', '["code"]', 0, 1, 0,
    (6, NULL, 5, 'Warehouse Import', 'Import warehouses', 'inventory', 'Warehouses', '["code"]', 0, 4, 0, 1, 1, S
    (7, NULL, 6, 'Warehouse Location Import', 'Import warehouse locations (bins, racks, zones)', 'inventory', 'Wa
    (8, NULL, 7, 'Chart of Accounts Import', 'Import chart of accounts with hierarchy and account types', 'financ
    (9, NULL, 8, 'Tax Code Import', 'Import tax codes with rates and linked accounts', 'finance', 'TaxCodes', '["
    (10, NULL, 9, 'Contact Import', 'Import contacts linked to customers or suppliers', 'core', 'Contacts', '["en
    (11, NULL, 10, 'Price List Item Import', 'Import price list items', 'inventory', 'PriceListItems', '["product
    (12, NULL, 11, 'Employee Import', 'Import employees', 'hr', 'Employees', '["employee_code"]', 0, 11, 0, 1, 1,
    (13, NULL, 12, 'Opening Stock Import', 'Import opening stock quantities and values per product per warehouse'
    (14, NULL, 13, 'Customer Opening Balance Import', 'Import outstanding customer invoices as opening AR balance
    (15, NULL, 14, 'Supplier Opening Balance Import', 'Import outstanding supplier bills as opening AP balances',
    (16, NULL, 15, 'GL Opening Balance Import', 'Import general ledger opening balances (debits must equal credit

SET IDENTITY_INSERT [import].[ImportProfiles] OFF;
```

M040.9 — Seed Import Profile Fields

> **Note:** Full field definitions for all 16 profiles are in **Appendix B**. The migration seeds approximately 200 rows into `import.ImportProfileFields` covering all standard fields for each importable entity type.

M040.10 — Create Import Feature Flag and Module Registration

sql

```
-- Register import module in module registry (Addendum 37)
INSERT INTO [system].[Modules]
    ([module_code], [name], [schema_name], [description],
     [is_core], [is_enabled_by_default], [sort_order])
VALUES
    ('IMPORT', 'Data Import', 'import',
     'Data import and migration framework — Excel/CSV upload, validation, and batch processing',
     1, 1, 170);

-- Feature flag
INSERT INTO [system].[FeatureFlags]
    ([module_code], [flag_code], [name], [description], [is_enabled_by_default])
VALUES
    ('IMPORT', 'DATA_IMPORT', 'Data Import', 'Enable data import and upload functionality', 1);
```

13. API Endpoints

13.1 Import Profile Endpoints

#	Method	Path	Auth	Description
1	GET	`/api/import/profiles`	`import.profile.view`	List import profiles for the org (system + custom)
2	GET	`/api/import/profiles/{id}`	`import.profile.view`	Get profile with field definitions
3	GET	`/api/import/profiles/{id}/template`	`import.session.upload`	Download Excel template for this profile
4	GET	`/api/import/migration-workbook`	`import.session.upload`	Download full migration workbook (all templates)

13.2 Import Session Endpoints

#	Method	Path	Auth	Description
5	POST	`/api/import/sessions`	`import.session.upload`	Upload file + create session. Multipart form: file + profile_id + duplicate_strategy
6	GET	`/api/import/sessions`	`import.session.upload`	List sessions for the org (paginated, sorted by created_at DESC)
7	GET	`/api/import/sessions/{id}`	`import.session.upload`	Get session details with counts and status
8	GET	`/api/import/sessions/{id}/preview`	`import.session.upload`	Get first 20 rows with mapped field values
9	GET	`/api/import/sessions/{id}/mappings`	`import.session.upload`	Get current column mappings
10	PUT	`/api/import/sessions/{id}/mappings`	`import.session.upload`	Update column mappings (user adjustments)
11	POST	`/api/import/sessions/{id}/mappings/save`	`import.session.upload`	Save current mapping as a named preset
12	POST	`/api/import/sessions/{id}/validate`	`import.session.execute`	Run validation pass
13	POST	`/api/import/sessions/{id}/execute`	`import.session.execute`	Start import execution
14	POST	`/api/import/sessions/{id}/cancel`	`import.session.execute`	Cancel in-progress import
15	GET	`/api/import/sessions/{id}/errors`	`import.session.upload`	Get error details (paginated)
16	GET	`/api/import/sessions/{id}/errors/download`	`import.session.upload`	Download error report as Excel
17	GET	`/api/import/sessions/{id}/progress`	`import.session.upload`	Get real-time progress (for background jobs)

13.3 API Request/Response Examples

POST `/api/import/sessions` — Upload File

```
Content-Type: multipart/form-data

Fields:
  file: <binary>
  profile_id: 3
  duplicate_strategy: 0    (0=Skip, 1=Update)
  sheet_name: "Sheet1"     (optional — defaults to first sheet)
  mapping_preset: "SAP Export" (optional — auto-load saved mapping)
```

Response (201 Created):

json


```
{
  "import_session_id": 142,
  "status": "Uploaded",
  "file_name": "products_export_2026.xlsx",
  "total_rows": 847,
  "sheet_names": ["Sheet1", "Sheet2"],
  "selected_sheet": "Sheet1",
  "auto_mappings": [
    {
      "source_column_index": 0,
      "source_column_name": "Item Code",
      "target_field_name": "sku",
      "target_display_name": "SKU / Code",
      "is_auto_mapped": true,
      "confidence_score": 0.90
    },
    {
      "source_column_index": 1,
      "source_column_name": "Item Description",
      "target_field_name": "name",
      "target_display_name": "Product Name",
      "is_auto_mapped": true,
      "confidence_score": 0.85
    }
  ],
  "unmapped_columns": ["Color", "Supplier Name"],
  "unmapped_required_fields": []
}
```


POST `/api/import/sessions/{id}/validate` — Response:

json

```
{
  "import_session_id": 142,
  "status": "Validated",
  "total_rows": 847,
  "valid_rows": 839,
  "invalid_rows": 8,
  "errors": [
    {
      "row_number": 5,
      "field_name": "category_code",
      "error_code": "LOOKUP_NOT_FOUND",
      "message": "Category 'GSKT' not found",
      "source_value": "GSKT"
    },
    {
      "row_number": 6,
      "field_name": "unit_code",
      "error_code": "LOOKUP_NOT_FOUND",
      "message": "Unit 'ML' not found. Available: PCS, KG, MTR, LTR, BOX",
      "source_value": "ML"
    }
  ],
  "error_count_by_code": {
    "LOOKUP_NOT_FOUND": 4,
    "DUPLICATE_IN_FILE": 2,
    "OUT_OF_RANGE": 1,
    "REQUIRED": 1
  }
}
```


14. Permission Claims

14.1 New Permissions

Permission	Display Name	Description
`import.profile.view`	View Import Profiles	View available import profiles and field definitions
`import.profile.manage`	Manage Import Profiles	Create, edit, and delete custom import profiles
`import.session.upload`	Upload Import Files	Upload files, map columns, download templates and error reports
`import.session.execute`	Execute Import	Run validation and execute data import
`import.session.manage`	Manage Import Sessions	Cancel, view all org imports (not just own)
`import.opening.balance`	Import Opening Balances	Import opening stock, customer, supplier, and GL balances

14.2 Default Role Mapping

Permission	Data Entry	Accountant	Manager	Admin
`import.profile.view`	✓	✓	✓	✓
`import.profile.manage`	—	—	—	✓
`import.session.upload`	✓	✓	✓	✓
`import.session.execute`	—	✓	✓	✓
`import.session.manage`	—	—	✓	✓
`import.opening.balance`	—	✓	—	✓

> **Rationale:** Data entry staff can upload and map files but cannot execute the actual import (prevents accidental large data changes). Accountants can execute and import opening balances. Only admins can manage import profiles.

15. Business Rules

15.1 File & Session Rules

Rule ID	Rule	Description
IMP-FILE-01	File type restriction	Only `.xlsx` and `.csv` files are accepted. `.xls` (pre-2007) is rejected with a message to save as `.xlsx`
IMP-FILE-02	File size limit	Files exceeding `max_file_size_mb` (default 10 MB) are rejected at upload
IMP-FILE-03	Row count limit	Files exceeding `max_rows_per_file` (default 50,000 rows) are rejected after parsing
IMP-FILE-04	Header row required	First row must contain column headers. A file with no detected header row is rejected
IMP-FILE-05	Empty file rejection	Files with 0 data rows (after header) are rejected
IMP-FILE-06	Sheet selection	For Excel files with multiple sheets, user selects the sheet. Default = first sheet
IMP-FILE-07	Session immutability	Once a session reaches `Importing` (status 4), its column mappings and duplicate strategy cannot be changed
IMP-FILE-08	Concurrent import block	Only one import session per entity type per org can be in `Importing` status at a time

15.2 Mapping Rules

Rule ID	Rule	Description
IMP-MAP-01	Required field coverage	All required fields must be mapped to a source column before validation can run. Unmapped required fields block the "Validate" button
IMP-MAP-02	Key field mandatory	At least one key field (is_key=true) must be mapped for duplicate detection to work
IMP-MAP-03	One-to-one mapping	A source column can map to at most one target field. A target field can receive from at most one source column
IMP-MAP-04	External ID auto-map	If a source column named "external_id", "ext_id", "legacy_id", or "source_id" exists, it is auto-mapped to the external_id field

15.3 Validation Rules

Rule ID	Rule	Description
IMP-VAL-01	All rows validated	Validation runs on every row. Partial validation is not supported
IMP-VAL-02	Validation is read-only	Validation checks the database (for lookups, duplicates) but writes nothing
IMP-VAL-03	Lookup case-insensitive	Lookup resolution is case-insensitive. "electronics" matches category code "ELECTRONICS"
IMP-VAL-04	Boolean parsing	Boolean fields accept: "true"/"false", "yes"/"no", "1"/"0", "y"/"n" (case-insensitive)
IMP-VAL-05	Date parsing	Date fields accept ISO 8601 (`2026-10-11`), common formats (`10/11/2026`, `11-Oct-2026`), and Excel date serial numbers
IMP-VAL-06	Decimal separator	Decimal fields accept both `.` and `,` as decimal separators. Thousand separators are stripped
IMP-VAL-07	Whitespace trimming	Leading and trailing whitespace is trimmed from all values before validation
IMP-VAL-08	Re-validation	Validation can be re-run after fixing and re-uploading the file to the same session (status resets to Mapping)

15.4 Execution Rules

Rule ID	Rule	Description
IMP-EXC-01	Only valid rows imported	Rows with status `Invalid` are skipped during execution. Only `Valid` rows are processed
IMP-EXC-02	Batch transaction	Each batch (default 100 rows) runs in a single database transaction. If any row in the batch fails, the entire batch rolls back
IMP-EXC-03	External reference registration	When a row has a non-empty `external_id` and is successfully imported, an ExternalReference record is created
IMP-EXC-04	Duplicate strategy enforcement	Skip = log and move on. Update = overwrite existing fields (null source values do not clear existing). CreateNew = always insert. Error = should not reach execution (caught in validation)
IMP-EXC-05	Audit trail	Every imported record has `created_by` set to the user who executed the import. Modified records have `modified_by` set accordingly
IMP-EXC-06	Background threshold	Sessions with > 500 valid rows are processed as Hangfire background jobs. Sessions ⩽ 500 rows process synchronously
IMP-EXC-07	Cancellation	A background import can be cancelled. Already-committed batches remain; only future batches are stopped. Status = Cancelled
IMP-EXC-08	Progress tracking	During background execution, progress is sent via SignalR: `ImportProgress` event with `{sessionId, processed, total}`

15.5 Opening Balance Rules

Rule ID	Rule	Description
IMP-OB-01	Opening balance permission	Opening balance imports (entity types 12–15) require the `import.opening.balance` permission
IMP-OB-02	GL balance equation	GL opening balance import requires total debits = total credits across all rows. The file is rejected at validation if unbalanced
IMP-OB-03	Single opening per entity	Opening stock: only one opening balance per product+warehouse combination. Re-import updates the existing balance
IMP-OB-04	Positive quantities	Opening stock quantities must be ⩾ 0. Negative opening stock is rejected
IMP-OB-05	Journal entry date	Opening balance journal entries use the organization's configured fiscal year start date
IMP-OB-06	No update on opening balances	Opening balance rows cannot be updated via re-import. Duplicate strategy is forced to `Error`. User must reverse and re-import

16. Test Scenarios

Test ID	Scenario	Expected Result	Category
TS-040-01	Upload valid .xlsx file with template headers	Session created, columns auto-mapped with confidence 1.0, status = Uploaded	Upload
TS-040-02	Upload valid .csv file	Session created, CSV parsed correctly, columns detected from header row	Upload
TS-040-03	Upload file exceeding max size (>10 MB)	Rejection: "File exceeds maximum size of 10 MB"	Upload
TS-040-04	Upload file exceeding max rows (>50,000)	Rejection: "File exceeds maximum of 50,000 rows"	Upload
TS-040-05	Upload .xls (old format) file	Rejection: "Please save as .xlsx format (Excel 2007+)"	Upload
TS-040-06	Upload file with no header row	Rejection: "No column headers detected in first row"	Upload
TS-040-07	Upload file with SAP-style headers ("MATNR", "MAKTX")	Auto-mapping via alias: MATNR → sku, MAKTX → name. Confidence = 0.90	Mapping
TS-040-08	Upload file with unmapped required fields	"Validate" button disabled. Message: "Map required field: Unit of Measure"	Mapping
TS-040-09	Save column mapping as preset "SAP Export"	Preset saved. Available in dropdown for next import of same entity type	Mapping
TS-040-10	Load saved mapping preset on new upload	Columns auto-mapped according to preset configuration	Mapping
TS-040-11	Validate file with all valid rows	All rows status = Valid. "Import N rows" button enabled	Validation
TS-040-12	Validate file with missing required field (blank SKU on row 5)	Row 5 invalid: "SKU / Code is required" (REQUIRED)	Validation
TS-040-13	Validate file with invalid data type (text in decimal field)	Row invalid: "Sale Price 'abc' is not a valid number" (INVALID_TYPE)	Validation
TS-040-14	Validate file with lookup not found (category "XYZ")	Row invalid: "Category 'XYZ' not found" (LOOKUP_NOT_FOUND). Suggest similar: "Did you mean 'XY'?"	Validation
TS-040-15	Validate file with duplicate key in file (same SKU on rows 3 and 45)	Row 45 invalid: "Duplicate SKU 'PRD-001' in file (row 3)" (DUPLICATE_IN_FILE)	Validation
TS-040-16	Validate file with duplicate key in DB (SKU already exists, strategy = Error)	Row invalid: "Product with SKU 'PRD-001' already exists in SMS" (DUPLICATE_IN_DB)	Validation
TS-040-17	Validate file with duplicate key in DB (strategy = Skip)	Row marked as Valid. During execution, row is skipped. Skipped count incremented	Validation

Test ID	Scenario	Expected Result	Category
TS-040-18	Download error report after validation	Excel file with original data + error columns. Failed rows highlighted in red	Report
TS-040-19	Import 50 products (below background threshold)	Synchronous import. All 50 records created. Status = Completed	Execution
TS-040-20	Import 1,000 products (above background threshold)	Hangfire job queued. Progress tracked via SignalR. Status → Importing → Completed	Execution
TS-040-21	Import with duplicate strategy = Update	Existing product found by SKU. Name, price updated. modified_by set. Row status = Updated	Execution
TS-040-22	Import with external_id for new record	Record created. ExternalReference registered: entity_type=Product, external_id="SAP-12345"	Execution
TS-040-23	Re-import with same external_id (strategy = Update)	Existing record found via ExternalReference. Record updated, not duplicated	Execution
TS-040-24	Cancel running background import	Future batches stopped. Already-committed batches preserved. Status = Cancelled	Execution
TS-040-25	Import opening stock (100 products × 3 warehouses)	300 StockMovement records created. StockLevels updated. 300 journal entries (DR Inventory, CR Equity)	Opening
TS-040-26	Import GL opening balance (balanced DR = CR)	Single journal entry created with all rows as lines. Entry date = fiscal year start	Opening
TS-040-27	Import GL opening balance (unbalanced DR ≠ CR)	Validation error: "Opening balances unbalanced. Debit: 150,000. Credit: 148,500. Difference: 1,500"	Opening
TS-040-28	Download template for Product Import	Excel file with headers, help row, sample row, category/unit dropdown validation	Template
TS-040-29	Download migration workbook	Single Excel file with 16 sheets in dependency order, each formatted as a template	Template
TS-040-30	Import products with category hierarchy (parent before child)	Parent categories created first (by sort_order in file). Child categories reference parent via parent_code lookup	Hierarchy

17. Development Phases

17.1 Phase Overview

Phase	Focus	Days	Key Deliverables
Phase 1	Import Schema + Profile Tables	3	M040.1–M040.3, M040.8–M040.9, ImportProfileService, API endpoints 1–2
Phase 2	Session + Mapping + Rows	4	M040.4–M040.6, ImportSessionService, ColumnMappingService, API endpoints 5–11
Phase 3	External References + Template Engine	3	M040.7, ExternalReferenceService, ImportTemplateService, API endpoints 3–4
Phase 4	Validation Engine	3	ImportValidationService, LookupResolverService, API endpoint 12
Phase 5	Import Execution Pipeline	4	ImportExecutionService, entity handlers (12 standard), Hangfire job, SignalR, API endpoints 13–17
Phase 6	Opening Balance Handlers	3	4 opening balance handlers, journal entry generation, stock movement creation
Phase 7	Import UI + Migration Wizard	4	Upload wizard (4 steps), import dashboard, migration wizard, error report download
Phase 8	Testing + Error Report	2	M040.10, E2E tests, error report Excel generation, edge case handling
Total		**26 days**	**10 migrations, 7 new tables, 17 endpoints, 6 permissions, 30 test scenarios**

17.2 Phase Dependencies

---

18. Future Enhancements

#	Feature	Description
1	Scheduled auto-import	SFTP/cloud storage integration for automated recurring imports (e.g., daily product price updates from supplier feeds)
2	API bulk import	REST API endpoint accepting JSON arrays for programmatic bulk insert (for ETL tools and integration platforms)
3	Custom entity import	Support importing into user-defined custom fields and custom entities
4	Image/document attachment import	ZIP upload containing product images mapped by SKU, or document attachments linked to imported records
5	Import undo/rollback	Full rollback of a completed import session — delete all created records and their external references
6	Multi-file relationship import	Upload a ZIP with related files (customers.xlsx + contacts.xlsx) that are processed together maintaining FK relationships
7	Legacy .xls support	Read old Excel format (.xls) using NPOI alongside ClosedXML
8	Diff preview	Before update imports, show a side-by-side diff of old vs. new values for each record being updated
9	Import scheduling	Schedule imports to run at specific times (e.g., "import this file at 2 AM when system load is low")
10	Google Sheets / OneDrive integration	Direct import from cloud spreadsheet URLs without file download+upload

19. Appendix A — Entity Relationship Summary

```
import.ImportProfiles (NEW)
  ├── M:1 → tenant.Organizations (nullable — NULL for system profiles)
  ├── M:1 → auth.Users (created_by — nullable for system)
  └── 1:M → import.ImportProfileFields (NEW)

import.ImportProfileFields (NEW)
  └── M:1 → import.ImportProfiles

import.ImportSessions (NEW)
  ├── M:1 → tenant.Organizations
  ├── M:1 → import.ImportProfiles
  ├── M:1 → auth.Users (created_by)
  ├── 1:M → import.ImportColumnMappings (NEW)
  │             └── M:1 → import.ImportProfileFields (target_field — nullable)
  ├── 1:M → import.ImportRows (NEW)
  └── 1:M → import.ExternalReferences (NEW, import_session_id — nullable)

import.ExternalReferences (NEW)
  ├── M:1 → tenant.Organizations
  └── M:1 → import.ImportSessions (nullable — manually registered refs)
```

---

20. Appendix B — Seeded Import Profile Fields

B.1 Customer Import (entity_type = 0)

field_name	display_name	type	required	key	sample
external_id	External ID	Text	—	—	CUST-SAP-001
code	Customer Code	Text	—	Yes	C001
name	Customer Name	Text	Yes	—	Acme Corporation
email	Email	Email	—	Yes	info@acme.com
phone	Phone	Phone	—	—	+1-555-0100
address_line1	Address Line 1	Text	—	—	123 Main Street
address_line2	Address Line 2	Text	—	—	Suite 400
city	City	Text	—	—	New York
state_province	State / Province	Text	—	—	NY
postal_code	Postal Code	Text	—	—	10001
country_code	Country Code (ISO)	Text	—	—	US
tax_number	Tax Number / VAT	Text	—	—	US12-3456789
credit_limit	Credit Limit	Decimal	—	—	50000.00
payment_terms_days	Payment Terms (days)	Integer	—	—	30
is_active	Active	Boolean	—	—	Yes

B.2 Supplier Import (entity_type = 1)

field_name	display_name	type	required	key	sample
external_id	External ID	Text	—	—	SUP-SAP-001
code	Supplier Code	Text	—	Yes	S001
name	Supplier Name	Text	Yes	—	Global Parts Inc.
email	Email	Email	—	Yes	orders@globalparts.com
phone	Phone	Phone	—	—	+44-20-7946-0958
address_line1	Address Line 1	Text	—	—	10 Industrial Lane
city	City	Text	—	—	London
country_code	Country Code (ISO)	Text	—	—	GB
tax_number	Tax Number / VAT	Text	—	—	GB123456789
payment_terms_days	Payment Terms (days)	Integer	—	—	45
currency_code	Currency (ISO)	Text	—	—	GBP
is_active	Active	Boolean	—	—	Yes

B.3 Product Import (entity_type = 2)

> See section 3.2 (C2) for full field definitions.

B.4 Product Category Import (entity_type = 3)

field_name	display_name	type	required	key	sample
external_id	External ID	Text	—	—	CAT-001
code	Category Code	Text	Yes	Yes	ELEC
name	Category Name	Text	Yes	—	Electronics
parent_code	Parent Category Code	Lookup	—	—	(empty for root)
description	Description	Text	—	—	Electronic components and devices
is_active	Active	Boolean	—	—	Yes

> **Hierarchy note:** Parent categories must appear before child categories in the file (or be pre-existing in SMS). The import processes rows in file order.

B.5 Unit of Measure (entity_type = 4)

field_name	display_name	type	required	key	sample
external_id	External ID	Text	—	—	UOM-001
code	Unit Code	Text	Yes	Yes	KG
name	Unit Name	Text	Yes	—	Kilogram
category	Category	Text	—	—	Weight
ratio	Conversion Ratio	Decimal	—	—	1.000
is_active	Active	Boolean	—	—	Yes

B.6 Warehouse (entity_type = 5)

field_name	display_name	type	required	key	sample
external_id	External ID	Text	—	—	WH-001
code	Warehouse Code	Text	Yes	Yes	MAIN-WH
name	Warehouse Name	Text	Yes	—	Main Warehouse
address	Address	Text	—	—	500 Storage Blvd
city	City	Text	—	—	Chicago
country_code	Country Code	Text	—	—	US
is_active	Active	Boolean	—	—	Yes

B.7 Chart of Accounts (entity_type = 7)

field_name	display_name	type	required	key	sample
external_id	External ID	Text	—	—	ACC-001
account_code	Account Code	Text	Yes	Yes	4100
name	Account Name	Text	Yes	—	Sales Revenue
account_type	Account Type	Text	Yes	—	Revenue
parent_code	Parent Account Code	Lookup	—	—	4000
currency_code	Currency	Text	—	—	USD
is_reconcilable	Is Reconcilable	Boolean	—	—	No
is_active	Active	Boolean	—	—	Yes

B.8 Opening Stock (entity_type = 12)

field_name	display_name	type	required	key	sample
product_sku	Product SKU	Lookup	Yes	Yes	PRD-001
warehouse_code	Warehouse Code	Lookup	Yes	Yes	MAIN-WH
location_code	Location Code	Lookup	—	—	A-01-03
quantity	Quantity	Decimal	Yes	—	150.000
unit_cost	Unit Cost	Currency	Yes	—	12.50
batch_number	Batch / Lot Number	Text	—	—	BATCH-2026-Q3
expiry_date	Expiry Date	Date	—	—	2027-06-30

B.9 GL Opening Balance (entity_type = 15)

field_name	display_name	type	required	key	sample
account_code	Account Code	Lookup	Yes	Yes	4100
debit_amount	Debit Amount	Currency	—	—	50000.00
credit_amount	Credit Amount	Currency	—	—	0.00
description	Description	Text	—	—	Opening balance — Sales Revenue

> **Note:** Remaining profiles (Warehouse Location, Tax Code, Contact, Price List Item, Employee, Customer Balance, Supplier Balance) follow the same pattern. Full field definitions are seeded by migration M040.9.

21. Appendix C — Import Flow Decision Matrix

```
Completed With Failed
Errors
```

End of FSD Addendum 40